

**IMPLEMENTASI *GRAPH RETRIEVAL*
AUGMENTED GENERATION UNTUK
MENINGKATKAN PRESISI RETRIEVAL DAN
VALIDITAS SINTAKSIS PADA KONFIGURASI
KUBERNETES**

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Mei 2026**

LEMBAR PENGESAHAN

IMPLEMENTASI *GRAPH RETRIEVAL AUGMENTED GENERATION* UNTUK MENINGKATKAN PRESISI RETRIEVAL DAN VALIDITAS SINTAKSIS PADA KONFIGURASI KUBERNETES

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 5 Mei 2026

Pembimbing

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP. 196808091991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 5 Mei 2026

Jihan Aurelia

NIM: 18222001

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 5 Mei 2026

Jihan Aurelia
NIM: 18222001

ABSTRAK

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

oleh

Jihan Aurelia

18222001

Transformasi menuju arsitektur *cloud-native* menempatkan Kubernetes sebagai standar *de facto* untuk orkestrasi kontainer, namun kompleksitas konfigurasi berbasis YAML menjadi tantangan operasional yang signifikan. Pemanfaatan *Large Language Models* (LLM) dalam menjawab pertanyaan teknis Kubernetes rentan menghasilkan konfigurasi yang keliru secara semantik atau tidak valid secara sintaksis—fenomena yang dikenal sebagai halusinasi. Pendekatan *Retrieval-Augmented Generation* (RAG) berbasis vektor yang umum digunakan juga terbukti tidak memadai karena gagal menangkap relasi hierarkis dan referensial yang diperlukan dalam domain Kubernetes, dengan akurasi yang hanya mencapai 63,4% *exact match* dan 69,8% *context precision*.

Penelitian ini membangun sistem *Graph Retrieval-Augmented Generation* (GraphRAG) untuk dokumentasi Kubernetes menggunakan metodologi CRISP-DM. *Knowledge graph* dibangun dari blok *definitions* pada spesifikasi OpenAPI Kubernetes v1.30 (*swagger.json*) dan menghasilkan 725 *node* definisi dengan 18 jenis *edge* yang merepresentasikan relasi struktural dan semantik antar-objek Kubernetes. Mekanisme *retrieval* bertingkat (*cascading retrieval*) memprioritaskan pencocokan nama eksak, dilengkapi *fallback* kemiripan vektor, serta penelusuran graf multi-lompatan (*multi-hop graph traversal*) hingga 3 lompatan. *Pipeline* respon dibangun menggunakan LangGraph dengan GPT-4o-mini untuk ekstraksi niat kueri dan Groq LLaMA untuk pembuatan jawaban.

Sistem dievaluasi menggunakan 97 *fixture* uji tervalidasi pakar dalam tiga dimensi: *Answer Quality* (AnsQ, bobot 40%), *Retrieval Quality* (RetQ, bobot 35%), dan *Reasoning Quality* (ReaQ, bobot 25%). Hasil evaluasi menunjukkan bahwa sistem GraphRAG mencapai skor AnsQ sebesar 0,58, RetQ sebesar 0,65, dan ReaQ sebesar 0,87 dengan total skor komposit sebesar 0,68. Sistem juga menghasilkan berkas konfigurasi YAML yang valid secara sintaksis dan dilengkapi jejak penalaran graf (*reasoning path*) sebagai mekanisme pendeteksi halusinasi LLM.

Kata kunci: *Graph Retrieval-Augmented Generation*, *knowledge graph*, Kubernetes, YAML, halusinasi LLM.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul "Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes". Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Jihan Aurelia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR PERSAMAAN	xxi
DAFTAR ALGORITMA	xxiii
DAFTAR <i>LISTING</i>	xxv
DAFTAR SIMBOL	xxvii
DAFTAR SINGKATAN	xxxix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	4
I.5 Metodologi	5
II STUDI LITERATUR	7
II.1 Arsitektur <i>Cloud</i> Modern dan Tantangan Kompleksitas	7
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi <i>Cloud-Native</i>	7
II.2.1 Gambaran Umum Arsitektur	8
II.2.2 <i>Resource Model</i> dan Hierarki	9
II.2.3 Manifes YAML dan <i>Infrastructure as Code</i>	10
II.3 <i>Large Language Models</i> (LLM)	11
II.3.1 Definisi dan Kapabilitas	11
II.3.2 <i>Prompt Engineering</i>	11
II.4 <i>Retrieval-Augmented Generation</i> (RAG)	12
II.4.1 Arsitektur <i>Retrieval-Augmented Generation</i>	12
II.4.2 Paradigma RAG	14
II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API	15
II.5 <i>Knowledge Graph</i> dan GraphRAG	15
II.5.1 Definisi dan Struktur	15
II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi	18
II.5.3 GraphRAG: Integrasi <i>Knowledge Graph</i> dan <i>Large Language Models</i>	18
II.6 Metrik Evaluasi Sistem Informasi Retrieval	19
II.6.1 Metrik Kualitas Jawaban	19
II.6.2 Metrik Kualitas Retrieval	20
II.6.3 Metrik Validitas Sintaksis YAML	21

III	ANALISIS MASALAH	23
III.1	Analisis Kondisi Saat Ini	23
III.1.1	Pemahaman Masalah Bisnis (<i>Business Understanding</i>)	23
III.1.2	Pemahaman Data (<i>Data Understanding</i>)	24
III.1.2.1	Sumber dan Spesifikasi Data (<i>Data Source & Specifications</i>)	24
III.1.2.2	Analisis Struktur Dokumen	24
III.1.2.3	Analisis Data Eksploratif Spesifikasi OpenAPI	25
III.1.2.4	Analisis Keterhubungan Data	27
III.1.2.5	Desain Taksonomi <i>Edge</i> untuk <i>Knowledge Graph</i> Kubernetes	28
III.1.2.6	Justifikasi Batas Kedalaman Penelusuran Graf (3 Lompatan)	30
III.1.2.7	Analisis Kompleksitas Pertanyaan dan Batasan Kemampuan Sistem	30
III.2	Analisis Kebutuhan	31
III.2.1	Kebutuhan Fungsional	33
III.2.2	Kebutuhan Non Fungsional	36
III.2.3	Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi	37
III.3	Alternatif Solusi	38
III.3.1	Hybrid KG–Vector RAG	38
III.3.2	<i>GNN-augmented GraphRAG</i> (G-Retriever)	39
III.3.3	<i>Hybrid Neural-Symbolic</i> dengan <i>Relational DB</i> (NeuSym-RAG)	39
III.4	Analisis Pemilihan Solusi	39
III.4.1	Rubrik Evaluasi Solusi	40
III.4.2	Matriks Keputusan & Justifikasi Penilaian	41
III.4.3	Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)	42
IV	PERANCANGAN	45
IV.1	Rancangan Solusi Secara Garis Besar	45
IV.2	Pemetaan Metodologi terhadap Kebutuhan Sistem	46
IV.3	Perancangan Komponen Sistem	52
IV.3.1	Pipeline <i>Ingestion</i> Data	52
IV.3.2	Perancangan <i>LangGraph Agent Pipeline</i>	54
IV.3.3	Perancangan Mekanisme <i>Retrieval</i> Bertingkat	55
IV.3.4	Perancangan Validasi YAML Tiga Lapis	56
IV.3.5	Perancangan Antarmuka Web Streamlit	57
V	IMPLEMENTASI	59
V.1	Lingkungan Implementasi	59
V.2	Implementasi <i>Ingestion</i> Data dan <i>Knowledge Graph</i>	61
V.3	Implementasi <i>Pipeline</i> LangGraph	62
V.4	Implementasi Mekanisme <i>Retrieval</i> Bertingkat	62
V.5	Implementasi Validasi YAML Tiga Lapis	63
V.6	Implementasi Antarmuka Pengguna	64

VI EVALUASI	65
VI.1 Metode Evaluasi	65
VI.2 Validasi Dataset oleh Pakar	66
VI.3 Hasil Evaluasi Kuantitatif	68
VI.3.1 Skor Keseluruhan	68
VI.3.2 Skor per Kategori <i>Fixture</i>	68
VI.3.3 Analisis Metrik <i>Reasoning Quality</i>	69
VI.4 Pembahasan Hasil Evaluasi	70
VI.5 Perbandingan dengan Pendekatan <i>Baseline</i>	71
VII PENUTUP	75
VII.1 Kesimpulan	75
VII.2 Saran	75
Lampiran A KODE PROGRAM	81
A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik	81
A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi	
Hasil Pengukuran Jarak	82
Lampiran B HASIL SURVEI LAPANGAN	83
B.1 Foto Survei Lokasi 1	83
B.2 Foto Survei Lokasi 2	83
B.3 Transkrip Wawancara dengan Narasumber	83

DAFTAR GAMBAR

I.1	Tahapan model referensi CRISP-DM (Niakšu 2015)	5
II.1	Arsitektur klaster Kubernetes yang terdiri atas <i>Control Plane</i> dan <i>Worker Nodes</i> , dihubungkan melalui <i>kube-apiserver</i> (The Kubernetes Authors 2024b)	8
II.2	Arsitektur <i>Knowledge Graph</i> (Yu dkk. 2021)	16
II.3	Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)	22
III.1	Proporsi objek Kubernetes <i>root</i> vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes	26
IV.1	Rancangan solusi berdasarkan metodologi CRISP-DM	45
IV.2	<i>Architecture Diagram</i> Solusi <i>GraphRAG</i> untuk Mitigasi Halusinasi pada Sistem RAG-LLM	50
IV.3	Diagram Urutan (<i>Sequence Diagram</i>) Alur Kerja Sistem <i>GraphRAG</i>	51

DAFTAR TABEL

II.1	Ringkasan kategori relasi antar-objek Kubernetes	10
III.1	Taksonomi 18 jenis <i>edge</i> dalam <i>knowledge graph</i> Kubernetes	29
III.2	Distribusi karakteristik <i>node</i> berdasarkan kedalaman penelusuran graf	30
III.3	Klasifikasi Tingkat Kompleksitas Pertanyaan Pengguna	31
III.4	<i>Technical Gap Analysis</i> pada Domain Kubernetes	32
III.5	Pemetaan Kebutuhan Bisnis terhadap <i>Technical Gap</i> dan Kebutuhan Fungsional	34
III.6	Kebutuhan Fungsional Sistem	34
III.7	Kebutuhan Non-Fungsional Sistem	37
III.8	Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi	38
III.9	Rubrik Penilaian Kriteria Evaluasi Solusi	40
III.10	Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi	41
IV.1	Pemetaan Tahap Metodologi	46
IV.2	Lima Fase Pipeline Ingestion Data	53
IV.3	Lima Node LangGraph Agent Pipeline	54
IV.4	Pemetaan <i>Intent Type</i> terhadap Kedalaman Traversal Graf	56
IV.5	Rancangan Tiga Lapis Validasi YAML	57
V.1	Dependensi Utama Implementasi Sistem GraphRAG Kubernetes	60
V.2	Statistik <i>Knowledge Graph</i> Kubernetes Hasil Konstruksi	61
VI.1	Profil Narasumber Wawancara Validasi Dataset	66
VI.2	Penilaian Realisme Sampel <i>Fixture</i> oleh Pakar (Skala 1–5)	67
VI.3	Skor Evaluasi per Dimensi dan Sub-Metrik (97 <i>Fixture</i>)	68
VI.4	Skor Evaluasi per Kategori <i>Fixture</i>	69
VI.5	Perbandingan Skor Evaluasi: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 <i>Fixture</i>)	72

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR *LISTING*

III.1 Injeksi ConfigMap	27
III.2 Injeksi Secret	27
A.1 Binary search code	81

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk <i>loss function</i> , tingkat signifikansi statistik	75
β	Koefisien bobot untuk <i>Binary Cross-Entropy loss</i>	75
λ	Koefisien bobot untuk CTC <i>loss</i> , parameter regularisasi	75
θ	Parameter model <i>neural network</i>	75
σ	Deviasi standar	75
μ	Rata-rata (<i>mean</i>)	75
ϵ	<i>Token blank</i> /kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	<i>Alignment path</i> dalam CTC	75
π_t	<i>Token</i> pada <i>timestep</i> t dalam <i>alignment path</i>	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi <i>loss</i> (kerugian)	75
$\mathcal{L}_{total}$	Total <i>loss function</i>	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	<i>Adversarial loss</i>	75
$\mathcal{L}_{reconstruction}$	<i>Reconstruction loss</i>	75
$\mathcal{L}_{L1}$	L1 <i>loss</i> (<i>Mean Absolute Error</i>)	75
$\mathcal{L}_{L2}$	L2 <i>loss</i> (<i>Mean Squared Error</i>)	75
$\mathcal{L}_{CTC}$	CTC (<i>Connectionist Temporal Classification</i>) <i>loss</i>	75
$\mathcal{L}_{perceptual}$	<i>Perceptual loss</i>	75

$\mathcal{L}_{pixel}$	<i>Pixel-wise loss</i>	75
$\mathcal{L}_{BCE}$	<i>Binary Cross-Entropy loss</i>	75
$\mathcal{L}_{GAN}$	<i>GAN loss</i>	75
$\mathcal{L}_{cycle}$	<i>Cycle consistency loss</i>	75
G	<i>Generator dalam GAN</i>	75
D	<i>Discriminator dalam GAN</i>	75
$G(z)$	<i>Output dari generator dengan input z</i>	75
$D(x)$	<i>Output dari discriminator dengan input x</i>	75
$G_{A \rightarrow B}$	<i>Generator dari domain A ke B</i>	75
$G_{B \rightarrow A}$	<i>Generator dari domain B ke A</i>	75
$V(D, G)$	<i>Fungsi nilai (value function) dalam GAN</i>	75
x	<i>Sampel data asli, input citra</i>	75
y	<i>Label atau ground truth, kondisi dalam conditional GAN</i>	75
z	<i>Noise vector atau representasi laten</i>	
$\mathbb{E}$	<i>Ekspektasi atau nilai harapan</i>	75
p_{data}	<i>Distribusi data asli</i>	75
p_z	<i>Distribusi prior (noise)</i>	75
$p(y x)$	<i>Probabilitas bersyarat output y given input x</i>	75
$p_t(\pi_t x)$	<i>Probabilitas token pada timestep t</i>	
$\mathcal{B}$	<i>Fungsi penciutan (collapse function) dalam CTC</i>	75
$\mathcal{B}^{-1}$	<i>Fungsi invers penciutan dalam CTC</i>	75
T	<i>Jumlah timestep atau panjang sequence</i>	
$ x - G(y) _1$	<i>L1 distance antara ground truth dan generated image</i>	75
$ x - G(y) _2$	<i>L2 distance antara ground truth dan generated image</i>	75
$N \times N$	<i>Ukuran patch dalam PatchGAN</i>	
d	<i>Cohen's d (effect size)</i>	75
n	<i>Jumlah sampel</i>	75
p	<i>p-value (nilai probabilitas statistik)</i>	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi industri teknologi informasi menuju arsitektur *cloud-native* kini menjadi kebutuhan fundamental, bukan sekadar tren. Gartner (2021) memproyeksikan bahwa pada tahun 2025, lebih dari 95% beban kerja digital akan di-*deploy* pada platform *cloud-native*. Angka ini meningkat tajam dari hanya 30% pada tahun 2021. Fondasi utama dari arsitektur *cloud-native* ini adalah penggunaan teknologi kontainer yang memungkinkan aplikasi dikemas dan dijalankan secara efisien. Namun, seiring dengan peningkatan skala sistem, pengelolaan ribuan kontainer secara manual tidak lagi memungkinkan sehingga dibutuhkan sebuah alat otomatisasi. Untuk menjembatani kebutuhan kompleks inilah, Kubernetes (K8s) hadir di pusat ekosistem dan mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer.

Berdasarkan laporan survei *Cloud Native Computing Foundation* (CNCF) tahun 2023, Kubernetes mendominasi pangsa pasar global karena ekosistemnya yang matang, dukungan komunitas yang luas, serta fleksibilitas dalam mengelola infrastruktur berskala besar secara deklaratif (Cloud Native Computing Foundation 2023). Pada tahun 2022, sekitar 58% pengguna telah menjalankan Kubernetes di lingkungan produksi dan 23% lainnya masih dalam tahap evaluasi (total 81%). Pada tahun 2023, angka tersebut meningkat menjadi 66% pengguna di produksi dengan 18% dalam evaluasi (total 84%). Peningkatan signifikan ini menunjukkan bahwa Kubernetes semakin menjadi fondasi utama dalam pengelolaan layanan *cloud-native* modern.

Meskipun Kubernetes mendominasi lanskap orkestrasi kontainer, platform ini menghadirkan tantangan operasional yang signifikan berupa kompleksitas konfigurasi berbasis berkas YAML. Paradigma deklaratif Kubernetes menuntut pengembang untuk menulis ratusan baris kode yang verbos dan hierarkis demi mendefinisikan *desired state* dari sebuah aplikasi yang mencakup berbagai spesifikasi parameter dari Kubernetes. Kompleksitas ini diperparah oleh interdependensi antar-objek yang

sangat ketat namun sering kali tidak terlihat secara eksplisit (*implicit dependencies*) yang mengakibatkan kesalahan kecil seperti ketidakcocokan *label selector* dapat memicu kegagalan sistem berantai. Akibat beban kognitif yang tinggi ini, kerumitan YAML tidak hanya menjadi hambatan skalabilitas utama bagi 65% penggunanya, tetapi juga memicu epidemi miskonfigurasi operasional dan keamanan; tercatat sekitar 80% insiden operasional berakar pada kompleksitas ini, dan 18% berkas konfigurasi sumber terbuka terbukti mengandung pelanggaran standar keamanan kritis industri (Rahman dkk. 2023).

Situasi tersebut berdampak langsung pada pemanfaatan *Large Language Models* (LLM) untuk menghasilkan kode maupun konfigurasi teknis Kubernetes. Walaupun model seperti GPT-4 menawarkan kemudahan dalam memahami dokumentasi API, kemampuan tersebut tidak selalu sejalan dengan ketepatan semantik. Studi evaluatif oleh Liu dkk. (2024) menunjukkan bahwa LLM dapat menghasilkan kode yang secara sintaksis tampak benar, tetapi keliru secara semantis. Dalam lingkungan *production-grade*, kesalahan semantik semacam ini tidak hanya bersifat minor, tetapi berpotensi memicu hilangnya layanan, gangguan orkestrasi, hingga *downtime* berskala besar.

Upaya mitigasi telah dilakukan melalui *Retrieval-Augmented Generation* (RAG) berbasis vektor untuk menambahkan konteks dokumentasi saat LLM menjawab pertanyaan teknis. Namun, penelitian oleh Wan dkk. (2025) mengungkapkan bahwa pendekatan vektor RAG hanya mengutamakan kesamaan *embedding* sehingga gagal menangkap relasi kausal, hierarkis, dan berbasis aturan teknis yang diperlukan untuk memahami konfigurasi domain Kubernetes. Pendekatan vektor RAG hanya mencapai 63,4% *exact match accuracy* dan 69,8% *context precision*, menunjukkan rendahnya ketepatan semantik dalam jawaban sistem. *Embedding* yang dilatih dari korpus umum juga mengabaikan terminologi teknis spesifik sehingga menghasilkan jawaban yang tampak relevan secara teks, tetapi tidak tepat secara domain.

Sebagai solusi, Wan dkk. (2025) mengusulkan penggabungan representasi pengetahuan terstruktur melalui *Knowledge Graph* (KG) untuk memberikan fondasi penalaran relasional yang eksplisit. Integrasi KG dalam sistem RAG menghasilkan peningkatan kinerja yang signifikan, mencapai 77,8% *exact match accuracy* dan 76,5% *context precision*, yaitu peningkatan lebih dari 14,4 poin persentase dibanding pendekatan vektor RAG. KG tidak hanya meningkatkan presisi pencarian informasi, tetapi juga memungkinkan *semantic alignment* melalui pembatasan ontologis sehingga model tidak hanya memilih teks yang mirip, tetapi juga mengakses

pengetahuan yang benar dalam domain.

Dengan demikian, pendekatan Graph-RAG menawarkan fondasi penalaran teknis yang lebih dapat dipertanggungjawabkan dan mampu menurunkan risiko halusinasi semantik berbasis domain, khususnya pada sistem *cloud-native* yang sensitif terhadap kesalahan konfigurasi. Berdasarkan urgensi tersebut, penelitian ini bertujuan untuk membangun sistem *Retrieval-Augmented Generation* (RAG) berbasis *Knowledge Graph* yang dikhususkan untuk dokumentasi Kubernetes. Pendekatan ini diharapkan mampu meningkatkan akurasi *retrieval* terhadap pertanyaan yang membutuhkan penalaran struktural dengan mengekspresikan relasi ontologis antar objek konfigurasi Kubernetes. Selain menghasilkan jawaban teknis berupa penjelasan dan konfigurasi YAML yang valid, sistem juga dirancang untuk memberikan jejak penalaran graf sebagai mekanisme pendeteksi halusinasi. Hal ini memungkinkan setiap keluaran tidak hanya relevan secara tekstual, tetapi juga dapat diverifikasi kesesuaiannya terhadap skema objek Kubernetes.

I.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab pertanyaan-pertanyaan berikut,

1. Bagaimana membangun *knowledge graph* dari spesifikasi Kubernetes guna merepresentasikan struktur dan relasi antar-objek konfigurasi secara komprehensif?
2. Bagaimana merancang mekanisme GraphRAG yang memanfaatkan representasi struktural dalam *knowledge graph* untuk meningkatkan presisi *retrieval* pada konfigurasi Kubernetes?
3. Bagaimana perbandingan kinerja antara GraphRAG, *Vanilla* RAG, dan *Vector-based* RAG dalam meningkatkan presisi *retrieval* serta validitas sintaksis berkas konfigurasi Kubernetes?

I.3 Tujuan

Berdasarkan rumusan masalah yang telah dijelaskan pada Bab I.2, tujuan dalam penulisan penelitian ini adalah sebagai berikut,

1. Membangun *knowledge graph* dari spesifikasi OpenAPI Kubernetes (swagger.json) melalui *pipeline* ekstraksi guna memetakan objek Kubernetes beserta relasi hierarkis dan referensial antar-objek Kubernetes.
2. Mengembangkan mekanisme GraphRAG dengan menelusuri jalur relasional (*graph traversal*) berdasarkan kueri pengguna, kemudian menginjeksikan *reasoning path* tersebut ke dalam *Large Language Model* (LLM) untuk menghasilkan jawaban konsisten sesuai logika domain Kubernetes.

3. Mengevaluasi kinerja metode GraphRAG menggunakan parameter *answer quality*, *retrieval quality*, dan *reasoning quality*, serta membandingkannya dengan *Vanilla* RAG maupun *Vector-based* RAG dalam meningkatkan presisi *retrieval* dan validitas sintaksis keluaran YAML menggunakan model generatif GPT-4o-mini.

I.4 Batasan Masalah

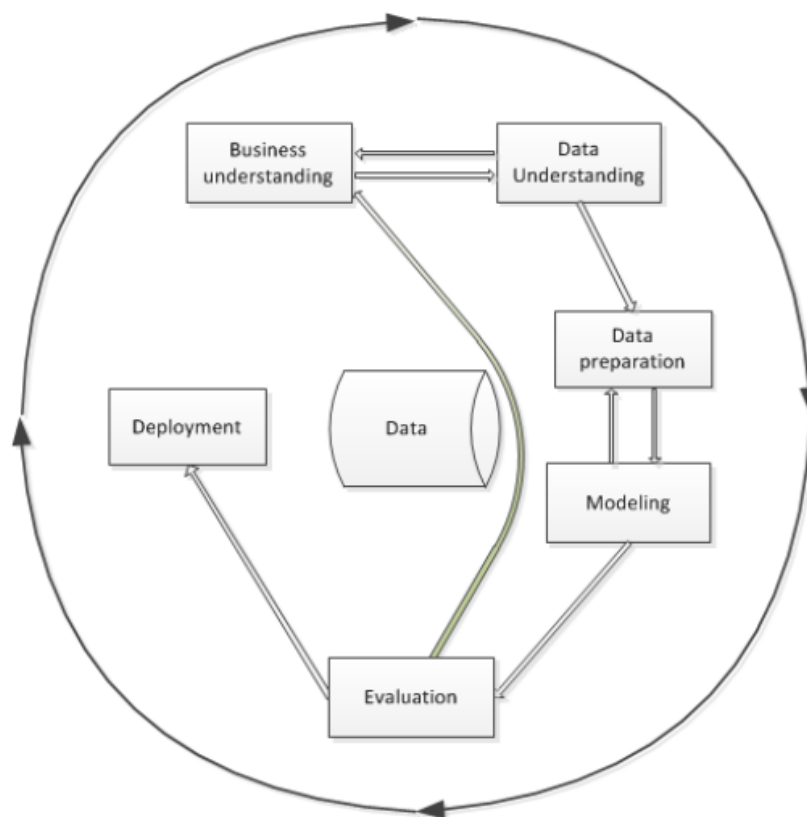
Agar penelitian ini tetap fokus dan *feasible* dalam lingkup Tugas Akhir, batasan-batasan berikut akan diterapkan,

1. Data penelitian bersumber secara eksklusif dari spesifikasi resmi Kubernetes OpenAPI (swagger.json) versi v1.30 tanpa melakukan *web scraping* dokumentasi HTML. Ekstraksi data dibatasi murni pada blok *definitions* dengan mengeliminasi blok operasional protokol API (seperti *paths*, *parameters*, dan *responses*) guna mencegah *context noise* pada LLM.
2. *Knowledge graph* yang dibangun hanya merepresentasikan struktur skema (*schema structure*) dan tidak mencakup perilaku operasional (*runtime behavior*) maupun logika validasi dengan fokus utama penelitian untuk menguji validitas struktur sintaksis YAML.
3. Luaran penelitian berupa manifes Kubernetes YAML murni (*plain YAML*) berdasarkan *OpenAPI Specification*, dilengkapi jejak penalaran graf (*graph reasoning path*) guna mendeteksi halusinasi LLM.
4. Luaran file YAML tidak diuji langsung pada kluster Kubernetes nyata (*actual cluster*). Pengujian sistem murni berfokus pada validitas sintaksis menggunakan pustaka *pyyaml* serta evaluasi kinerja berdasarkan dataset uji.
5. Model LLM yang digunakan berasal dari *pre-trained models* yang diakses melalui API (GPT-4o-mini) tanpa proses *fine-tuning*.
6. Algoritma *graph traversal* dibatasi maksimal 3 lompatan (*hops*) sesuai analisis distribusi kedalaman skema OpenAPI Kubernetes. Kedalaman 1–2 menghasilkan *node* spesifik *resource* yang hanya dibagikan oleh 2–3 *resource*, kedalaman 3 mencapai *PodSpec*, sedangkan *field* tingkat kontainer (kedalaman 4) tidak diambil karena didominasi tipe generik bersama, sedangkan kedalaman 4+ didominasi tipe generik (*Quantity*, *IntOrString*, *LocalObjectReference*) yang dibagikan oleh 19–136 *resource* sehingga menurunkan presisi konteks. Pembatasan ini bertujuan mengoptimalkan *Context Precision* serta efisiensi beban komputasi pada sumber daya lokal.
7. Proses pengolahan dan pra-pemrosesan data dibatasi pada lingkungan **Google Colaboratory** dan **Jupyter Notebook**.
8. Kinerja komputasi dan hasil eksperimen didasarkan pada perangkat keras spe-

sifik (PC dengan CPU AMD Ryzen 5, GPU AMD Radeon RX 6750XT, RAM 16 GB) dan mungkin bervariasi pada konfigurasi lain.

I.5 Metodologi

Pelaksanaan Tugas Akhir ini mengadopsi kerangka kerja *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang dapat dilihat pada Gambar I.1 sebagai metodologi utama. CRISP-DM dipilih karena memberikan proses terstruktur, iteratif, dan dapat diadaptasi untuk pengembangan sistem berbasis analisis data dan representasi pengetahuan (Niakšu 2015). Tahapan metodologi yang digunakan terdiri dari enam fase berikut,



Gambar I.1 Tahapan model referensi CRISP-DM (Niakšu 2015)

1. Pemahaman Bisnis (*Business Understanding*)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan serta masalah inti yang ingin diselesaikan melalui pemanfaatan data. Dalam konteks penelitian ini, permasalahan yang diangkat adalah ketidaktepatan jawaban teknis yang dihasilkan *Large Language Models* (LLM) ketika merujuk dokumentasi Kubernetes, terutama pada konfigurasi YAML yang bersifat struktural dan rentan mengandung halusinasi. Oleh karena itu, kebutuhan penelitian diarahkan pada peningkatan akurasi penalaran struktural pada proses *retrieval* serta mitigasi halusinasi konfigurasi pada sistem *cloud-native*.

2. Pemahaman Data (*Data Understanding*)

Fase ini berfokus pada pengumpulan dan eksplorasi sumber data untuk memahami struktur, kualitas, serta batasannya. Pada penelitian ini, data diperoleh dari spesifikasi resmi Kubernetes OpenAPI yang dianalisis untuk memahami jenis objek Kubernetes, atribut yang dimiliki, serta relasi hierarkis dan referensial antar-objek Kubernetes. Pemahaman ini menentukan objek Kubernetes yang akan dimodelkan dalam *knowledge graph*.

3. Persiapan Data (*Data Preparation*)

Tahapan ini mencakup pembersihan dan transformasi data agar siap digunakan dalam proses pemodelan. Dalam penelitian ini, dilakukan ekstraksi berbasis *script* dari spesifikasi OpenAPI Kubernetes (JSON) untuk membentuk representasi entitas, yaitu relasi yang dapat dimodelkan sebagai *Knowledge Graph*. Proses ini mencakup identifikasi objek, pemetaan hubungan antar-objek, serta penghubungan *cross-resource references*. Hasil akhirnya berupa struktur graf yang siap digunakan pada mekanisme *retrieval*.

4. Pemodelan (*Modeling*)

Fase ini bertujuan untuk menerapkan algoritma yang sesuai guna menyelesaikan masalah yang telah dirumuskan. Pada penelitian ini, dibangun mekanisme *graph-guided retrieval* yang menelusuri graf menggunakan *graph traversal* berdasarkan kueri pengguna dan menghasilkan *reasoning path*. Jalur penalaran tersebut kemudian diinjeksikan sebagai konteks pada proses RAG untuk memandu LLM dalam menghasilkan jawaban serta konfigurasi YAML yang sesuai dengan logika sistem Kubernetes.

5. Evaluasi (*Evaluation*)

Tahap evaluasi bertujuan menilai kualitas model dan membandingkannya dengan pendekatan alternatif menggunakan metrik yang terukur. Dalam penelitian ini, kinerja sistem dievaluasi menggunakan metrik *answer quality*, *retrieval quality*, dan *reasoning quality*, kemudian dibandingkan dengan dua *baseline*, yaitu *Vanilla LLM* dan *Vector-based RAG*.

6. Penyebaran (*Deployment*)

Fase ini berkaitan dengan penyajian hasil implementasi dalam bentuk prototipe dan dokumentasi. Pada penelitian ini, penyebaran diwujudkan dalam bentuk antarmuka web interaktif berbasis Streamlit yang menampilkan jawaban, konfigurasi YAML, serta *reasoning path* sebagai bukti penalaran graf. Selain itu, dokumentasi lengkap disusun sebagai bentuk penyebaran akademik yang memuat rancangan sistem, hasil evaluasi, serta rekomendasi penelitian selanjutnya.

BAB II

STUDI LITERATUR

II.1 Arsitektur *Cloud* Modern dan Tantangan Kompleksitas

Pergeseran paradigma dari arsitektur monolitik menuju layanan mikro (*microservices*) kini telah menjadi fondasi utama dalam pengembangan aplikasi *cloud-native* modern. Berbeda dengan pendekatan arsitektur tradisional, gaya arsitektur layanan mikro mendistribusikan logika bisnis secara masif ke dalam puluhan hingga ratusan layanan kecil independen. Distribusi fungsionalitas ini memunculkan kebutuhan akan mekanisme penerapan (*deployment*) mandiri. Sebagai solusi, layanan mikro secara alami terintegrasi dengan platform berbasis kontainer karena setiap layanan dapat dikemas dan didistribusikan di dalam kontainernya masing-masing (Soldani, Tamburri, dan Van Den Heuvel 2018). Walaupun memberikan keuntungan operasional yang signifikan berupa portabilitas lintas *cloud*, skalabilitas tingkat tinggi, dan kebebasan pemilihan teknologi bagi pengembang, ekosistem kontainer ini memunculkan paradoks kompleksitas baru (Soldani, Tamburri, dan Van Den Heuvel 2018).

Dalam lingkungan terdistribusi ini, setiap layanan berkomunikasi melalui antarmuka API yang bertindak sebagai kontrak komunikasi esensial antar-layanan (Soldani, Tamburri, dan Van Den Heuvel 2018). Pemahaman kognitif yang keliru terhadap arsitektur dan kontrak API ini sering kali berujung pada kegagalan operasional; kegagalan satu layanan dapat memicu rentetan kegagalan layanan lain yang bergantung padanya (*cascading failures*) (Soldani, Tamburri, dan Van Den Heuvel 2018). Akibatnya, dokumentasi API dan manifes konfigurasi orkestrasi mengalami lonjakan kompleksitas yang menjadi tantangan utama pengembang *cloud-native*.

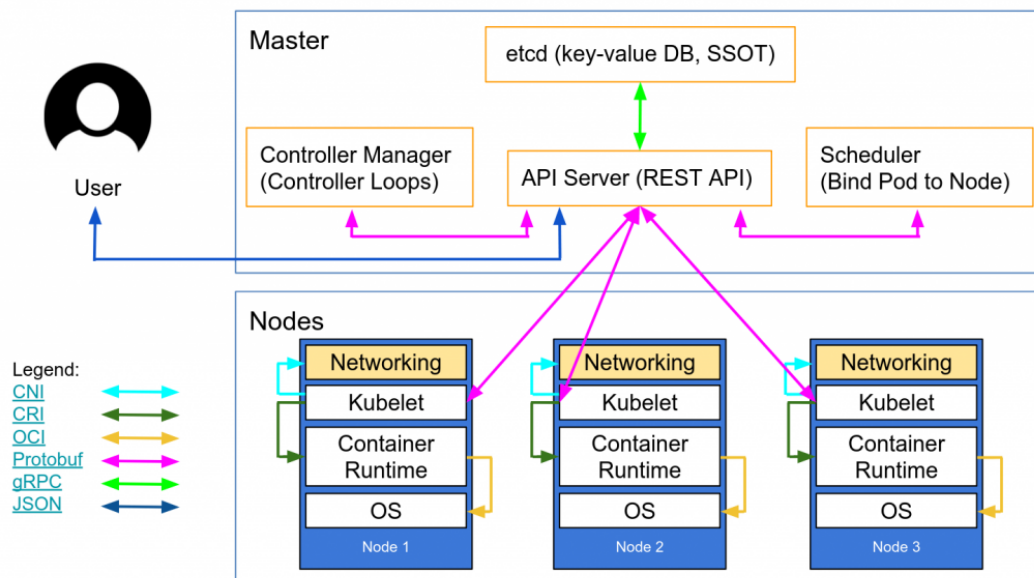
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi *Cloud-Native*

Kubernetes merupakan platform orkestrasi kontainer yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan manajemen aplikasi berbasis kontainer dalam lingkungan *cloud-native* (The Kubernetes Authors 2024b). Platform ini menyediakan mekanisme deklaratif berbasis konfigurasi yang memungkinkan pengguna mendefinisikan keadaan sistem yang diinginkan (*desired state*) melalui berkas

konfigurasi, sementara Kubernetes menjaga agar keadaan tersebut tetap terpenuhi melalui rangkaian komponen pengendali (*controller loops*).

II.2.1 Gambaran Umum Arsitektur

Arsitektur Kubernetes dibangun atas konsep kluster yang terdiri dari dua kelompok komponen utama, yaitu *Control Plane* dan *Worker Nodes*. *Control Plane* berfungsi sebagai pengendali utama yang membuat keputusan bisnis aplikasi, misalnya penjadwalan, pemantauan, serta konsistensi status. *Worker Nodes* merupakan entitas komputasi fisik/virtual yang menjalankan kontainer sebagai satuan aplikasi. Kedua komponen ini berkomunikasi secara internal melalui API server yang menjadi pintu utama seluruh permintaan sistem.



Gambar II.1 Arsitektur kluster Kubernetes yang terdiri atas *Control Plane* dan *Worker Nodes*, dihubungkan melalui *kube-apiserver* (The Kubernetes Authors 2024b)

1. *Control Plane*

Control Plane bertanggung jawab mengelola keadaan (*state*) kluster secara keseluruhan. Komponen utamanya meliputi,

- kube-apiserver** sebagai gerbang akses seluruh operasi manipulasi objek konfigurasi melalui API,
- etcd** sebagai basis data *key-value* yang menyimpan informasi konfigurasi dan status,
- kube-scheduler** yang menentukan Node penempatan Pod berdasarkan ketersediaan sumber daya,

- d. **kube-controller-manager** yang berisi sekumpulan kontroler untuk menjaga kesesuaian antara kondisi aktual dan *desired state*. Melalui mekanisme deklaratif ini, Kubernetes secara otomatis memperbaiki status objek yang tidak sesuai dan memastikan setiap aplikasi berjalan sesuai aturan konfigurasi yang ditentukan.

2. *Worker Nodes*

Worker Nodes merupakan lapisan eksekusi tempat aplikasi berjalan dalam bentuk Pod. Node terdiri atas tiga komponen inti:

- a. **kubelet** yang bertugas mengeksekusi Pod sesuai *specification* yang diterima dari *Control Plane*,
- b. **kube-proxy** yang menyediakan layanan *network forwarding* dan *load balancing* sederhana antar Pod dalam klaster,
- c. **container runtime** yang bertanggung jawab menjalankan kontainer. Dengan demikian, Node menjalankan komputasi terdistribusi yang dikontrol oleh *Control Plane* melalui komunikasi berbasis API.

II.2.2 *Resource Model dan Hierarki*

Dalam *resource model* Kubernetes, terdapat beberapa objek inti pembangun aplikasi yang paling sering digunakan:

- a. **Pod**: Unit komputasi terkecil sebagai pembungkus kontainer aplikasi
- b. **Deployment**: Pengatur replika dan proses pembaruan versi untuk mencapai ketersediaan tinggi
- c. **Service**: Penyedia titik akses jaringan yang stabil untuk aplikasi di dalam *Pod*
- d. **Ingress**: Pengatur rute lalu lintas eksternal masuk ke dalam klaster
- e. **ConfigMap**: Penyimpan variabel lingkungan dan konfigurasi non-sensitif
- f. **Secret**: Pengelola data sensitif seperti kredensial dan token
- g. **PersistentVolumeClaim**: Permintaan penyimpanan persisten untuk data yang harus bertahan
- h. **ServiceAccount**: Identitas yang digunakan oleh *Pod* untuk berinteraksi dengan API server
- i. **Role** dan **RoleBinding**: Kontrol akses berbasis peran (RBAC)
- j. **HorizontalPodAutoscaler**: Pengatur skala otomatis berdasarkan metrik penggunaan

Antar-objek ini saling terikat melalui berbagai jenis relasi semantik yang dapat dikategorikan ke dalam 7 kelompok. Ringkasan kategori relasi ditampilkan pada Tabel II.1, sedangkan desain taksonomi *edge* secara rinci disajikan pada Bab III sebagai bagian dari kontribusi penelitian.

Tabel II.1 Ringkasan kategori relasi antar-objek Kubernetes

Kategori	Jumlah <i>Edge</i>	Contoh <i>Edge</i>
Struktural	4	HAS_PROPERTY, EXTENDS
<i>Workload</i>	3	CONTAINS_POD_TEMPLATE
Penyimpanan	3	CLAIMS_VOLUME, MOUNTS_VOLUME
Konfigurasi	2	LOADS_CONFIGMAP, USES_SECRET
Jaringan	2	SELECTS_POD, ROUTES_TO_SERVICE
RBAC	3	BINDS_ROLE, USES_SERVICE_ACCOUNT
<i>Autoscaling</i>	1	SCALES_RESOURCE
Total	18	

Pemetaan relasi kepemilikan (*ownership*) dan ketergantungan fungsional (*dependencies*) inilah yang akan secara langsung dikonversi menjadi skema graf pengetahuan (*knowledge graph*) pada penelitian ini. Sebagai contoh, sebuah *Deployment* secara hierarkis memiliki sub-sumber daya *PodSpec* melalui relasi CONTAINS_POD_TEMPLATE, dan *Pod* tersebut memiliki ketergantungan operasional terhadap *ConfigMap* tertentu melalui relasi LOADS_CONFIGMAP. Total terdapat 18 jenis *edge* yang terdistribusi dalam 7 kategori relasi sebagaimana dirangkum pada Tabel II.1. Desain taksonomi *edge* secara rinci dibahas pada Bab III.

II.2.3 Manifes YAML dan *Infrastructure as Code*

Pengembang berinteraksi dengan model sumber daya Kubernetes melalui pendekatan *Infrastructure as Code* (IaC) (The Kubernetes Authors 2024a). Pendekatan ini mewajibkan pengguna untuk mendeklarasikan wujud infrastruktur yang diinginkan (*desired state*) ke dalam bentuk dokumen teks berformat YAML. Setiap manifes YAML standar wajib memiliki empat atribut struktural utama agar dapat diterima oleh API Kubernetes:

1. *apiVersion*: Menentukan versi skema API pembungkus objek.
2. *kind*: Mendefinisikan jenis objek Kubernetes.
3. *metadata*: Memuat data identifikasi unik seperti nama, label pengelompokan, dan *namespace*.
4. *spec*: Berisi spesifikasi teknis rinci penentu sifat dan bentuk hierarki objek tersebut.

Penyusunan manifes YAML menuntut pemahaman terkait batasan skema. Pengembang harus bisa membedakan secara presisi antara ruas wajib (*required*) penentu validitas dan ruas opsional (*optional*) penambah fitur khusus. Praktik terbaik (*best practices*) dalam penulisan IaC menyarankan modularitas dokumen, konsistensi penggunaan label, dan pemisahan logika aplikasi dari data konfigurasi. Sebuah struktur manifes YAML yang mematuhi kelengkapan ruas wajib (*field requirements*) dan valid secara sintaksis inilah yang menjadi representasi target luaran yang akan dibangkitkan oleh sistem generasi pada penelitian ini.

II.3 Large Language Models (LLM)

II.3.1 Definisi dan Kapabilitas

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis jaringan saraf berskala besar yang dilatih pada korpus teks *multi-domain* untuk mempelajari representasi bahasa alami secara mendalam (Wan dkk. 2025). Melalui proses pelatihan tersebut, LLM mampu menginterpretasikan konteks linguistik yang kompleks serta menghasilkan keluaran berupa teks yang koheren, kontekstual, dan adaptif terhadap berbagai jenis tugas seperti *question answering*, penalaran, serta peringkasan teks.

Meskipun unggul dalam pemahaman bahasa alami, LLM memiliki keterbatasan faktual yang signifikan ketika diterapkan pada domain teknis, terutama terhadap pengetahuan yang presisi, bersifat prosedural, serta memerlukan landasan ontologis. Beberapa keterbatasan yang disorot oleh Wan dkk. (2025) meliputi,

- a. *Hallucination*, yaitu kemampuan menghasilkan jawaban yang secara linguistik meyakinkan tetapi tidak memiliki landasan fakta yang jelas sehingga berisiko menyesatkan proses pengambilan keputusan teknis.
- b. Keterbatasan cakupan pengetahuan, LLM hanya mengandalkan data pelatihan awal sehingga tidak selalu mencakup informasi domain yang mutakhir.
- c. Ketidadaan atribusi sumber, yaitu LLM tidak dapat memberikan justifikasi atau referensi eksplisit terhadap jawaban yang dihasilkan.

Keterbatasan tersebut menjadi alasan utama perlunya integrasi antara LLM dan arsitektur *retrieval-augmented generation* (RAG).

II.3.2 Prompt Engineering

Prompt engineering merupakan teknik perancangan instruksi masukan bagi LLM untuk mengendalikan keluaran model berdasarkan tujuan tugas, cakupan konteks, serta batasan informasi yang diizinkan (Wan dkk. 2025). Dalam domain teknis, teknik ini tidak hanya berperan sebagai instruksi linguistik, tetapi juga sebagai rep-

representasi pengetahuan eksplisit yang mendefinisikan bagaimana LLM harus memanfaatkan sumber informasi domain.

Efektivitas keluaran LLM dipengaruhi secara signifikan oleh strategi *prompt engineering* berikut,

- a. Spesifikasi tugas yang eksplisit, termasuk definisi peran model dan batasan prosedural, misalnya instruksi untuk membatasi jawaban pada proses manufaktur tertentu.
- b. Penyediaan konteks terstruktur, melalui penyisipan entitas domain, hubungan semantik, serta cuplikan dokumen terkait yang relevan dengan pertanyaan.
- c. Pemberian batasan (*constraint injection*) yang secara eksplisit membatasi ruang solusi berdasarkan pengetahuan terbatas dalam dokumen yang valid.
- d. Penyusunan format jawaban, misalnya dalam bentuk tabel, daftar berhierarki, atau penjelasan berbasis pembuktian prosedural.

Strategi tersebut tidak hanya mengontrol keluaran, tetapi juga mencegah LLM menafsirkan konteks di luar pengetahuan domain sehingga berperan sebagai mekanisme mitigasi terhadap fenomena *hallucination*.

II.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) merupakan arsitektur pemrosesan bahasa alami yang mengombinasikan mekanisme *Information Retrieval* (IR) dengan kemampuan generatif *Large Language Models* (LLM). Pendekatan ini dikembangkan untuk mengatasi keterbatasan memori parametris pada model bahasa, khususnya dalam menangani tugas yang bersifat *knowledge-intensive*, yaitu model rentan menghasilkan informasi yang keliru atau tidak berbasis fakta (*hallucination*) (Antonio Moreno-Cediel 2025).

Berbeda dengan metode generatif konvensional yang hanya mengandalkan pengetahuan tersimpan selama proses pelatihan, RAG memanfaatkan basis pengetahuan non-parametris melalui proses pencarian (*retrieval*) sehingga jawaban yang dihasilkan lebih akurat dan relevan terhadap konteks masukan.

II.4.1 Arsitektur Retrieval-Augmented Generation

Arsitektur *Retrieval-Augmented Generation* (RAG) merupakan pendekatan hibrida yang menggabungkan kemampuan penalaran *Large Language Models* (LLM) dengan basis pengetahuan eksternal melalui proses pencarian semantik. Struktur ini dirancang untuk mengurangi *hallucination*, meningkatkan akurasi fakta, serta mendukung pembaruan pengetahuan secara dinamis (Gao dkk. 2024). Secara umum, alur kerja RAG terdiri atas empat tahapan teknis utama yang berurutan dan saling

bergantung, yaitu *Data Ingestion & Indexing*, *Retrieval*, *Augmentation*, dan *Generation*.

1. *Data Ingestion & Indexing*

Tahapan ini bersifat *offline* dan mencakup lima langkah berurutan: (a) *akuisisi data* dari sumber dokumentasi teknis atau spesifikasi API, (b) *preprocessing* berupa normalisasi format dan ekstraksi metadata, (c) *chunking* untuk memecah dokumen menjadi unit 200–500 token, (d) *embedding* menggunakan model seperti MiniLM atau BERT untuk mengubah setiap *chunk* menjadi representasi vektor, serta (e) *indexing* ke *vector store* (misalnya FAISS atau Chroma) agar pencarian berbasis kedekatan semantik dapat dilakukan secara efisien.

2. *Retrieval*

Proses *retrieval* dilakukan ketika pengguna mengajukan kueri. Sistem akan mencari konteks paling relevan melalui mekanisme berikut,

a. *Embedding Kueri*

Kueri pengguna diubah menjadi vektor menggunakan model *embedding* yang sama dengan proses *indexing*.

b. *Similarity Search*

Dilakukan pencocokan vektor menggunakan metrik jarak seperti *cosine similarity*, *dot-product*, atau *Euclidean distance*.

c. *Top-k Selection*

Sistem memilih sejumlah dokumen teratas (umumnya $k = 3$ atau $k = 5$) yang memiliki skor kemiripan tertinggi dan mengandung konteks yang berpotensi menjawab pertanyaan.

Kualitas *retrieval* yang baik menjadi faktor kunci dalam menurunkan risiko *hallucination* pada LLM karena jawaban dibatasi oleh fakta yang relevan.

3. *Augmentation*

Tahap *augmentation* bertugas menyisipkan konteks hasil *retrieval* ke dalam *prompt* secara sistematis sehingga dapat dipahami oleh LLM. Prosesnya mencakup tahapan berikut,

a. *Concatenation*

Sistem menggabungkan pertanyaan pengguna dengan konten dokumen yang ditemukan.

b. *Formatting*

Struktur informasi dapat diberikan dalam format teks baku, tabel, *JSON schema*, *instruction-style*, atau bentuk terstruktur lainnya.

c. *Constraint Injection*

Sistem memberikan batasan eksplisit, misalnya “*jawab hanya menggunakan konteks yang diberikan, jangan menambah informasi baru*”, untuk mencegah LLM berhalusinasi.

II.4.2 Paradigma RAG

Retrieval-Augmented Generation berkembang dalam tiga paradigma utama, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*. Ketiga paradigma ini mencerminkan evolusi dari *pipeline* sederhana menuju arsitektur yang dapat diadaptasi untuk berbagai kebutuhan, mulai dari peningkatan akurasi sampai fleksibilitas sistem (Gao dkk. 2024).

1. *Naive RAG*

Paradigma ini merupakan bentuk paling dasar dari RAG. *Pipeline* umumnya terdiri dari tiga langkah, yaitu *indexing*, *retrieval*, dan *generation*. Dokumen melalui proses *chunking*, kemudian direpresentasikan menggunakan *embedding* dan disimpan pada *vector store*. Ketika pertanyaan diajukan, sistem mengambil k dokumen paling relevan menggunakan pencarian berbasis kesamaan semantik, lalu model generatif menyusun jawaban berdasarkan konteks tersebut. Keterbatasan utama pendekatan ini adalah ketergantungan penuh terhadap kualitas *chunk* dan hasil *retrieval* yang dapat mengarah pada jawaban yang tidak akurat.

2. *Advanced RAG*

Paradigma ini melakukan optimasi pada dua fase utama, yaitu *pre-retrieval* dan *post-retrieval*. Tahap *pre-retrieval* meliputi peningkatan kualitas indeks seperti pengayaan metadata, peningkatan granularitas *chunk*, serta teknik *query rewriting* dan *query expansion* untuk meningkatkan relevansi hasil pencarian. Sementara itu, *post-retrieval* dilakukan melalui *reranking* serta *context compression* agar konteks yang terkirim ke model lebih berfokus pada informasi penting saja. Paradigma ini masih mengikuti alur linear tetapi menghasilkan akurasi yang lebih tinggi dibanding *Naive RAG*.

3. *Modular RAG*

Paradigma ini memperluas arsitektur RAG menjadi sistem yang dapat dikustomisasi melalui penambahan atau penggantian modul seperti *Search*, *Memory*, *Routing*, dan *Task Adapter*. Modular RAG juga memungkinkan pola *iterative* dan *adaptive retrieval* sehingga menjadikannya paradigma paling fleksibel untuk aplikasi domain spesifik.

II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API

Dokumentasi Web API, termasuk Kubernetes, memiliki karakteristik unik berupa kombinasi deskripsi sintaks terstruktur (misalnya skema JSON/YAML) dan penjelasan semantik dalam bentuk teks alami. Penelitian Kotstein dan Decker (2024) menunjukkan bahwa pemrosesan semantik terhadap dokumentasi API dapat dilakukan tanpa ontologi formal dengan memetakan makna berdasarkan nama parameter, struktur hierarkis, dan deskripsi bahasa alami.

Dalam konteks Kubernetes, komponen API seperti Pod, Service, dan Deployment direpresentasikan melalui skema bertingkat yang bersifat kompleks dan memiliki ketergantungan semantik. Oleh karena itu, implementasi RAG pada domain ini membutuhkan,

1. ***Schema-aware semantic chunking***

Teknik ini memastikan bahwa pemotongan dokumen (*chunking*) dilakukan mengikuti struktur skema objek API. Setiap potongan teks harus mewakili satu entitas atau blok atribut yang utuh sehingga relasi antar-*field* tidak terpisah atau kehilangan konteks. Dengan demikian, representasi vektor yang dihasilkan tetap mempertahankan hierarki objek.

2. ***Path-based serialization***

Teknik ini menyimpan setiap elemen API beserta jalur lengkapnya dalam struktur objek sehingga posisi semantiknya tetap terpelihara.

3. ***Hybrid index retrieval***

Pendekatan ini mengombinasikan pencarian berbasis kesamaan semantik teks dengan pembobotan struktur sintaksis dari objek API. Selain memanfaatkan *embedding* untuk menemukan kemiripan makna antar-teks, sistem juga memberikan prioritas pada elemen yang memiliki signifikansi struktural (misalnya *field* wajib atau relasi antar-objek). Hasilnya, mekanisme *retrieval* menghasilkan konteks yang tidak hanya relevan secara linguistik, tetapi juga benar secara hierarki dan fungsi API.

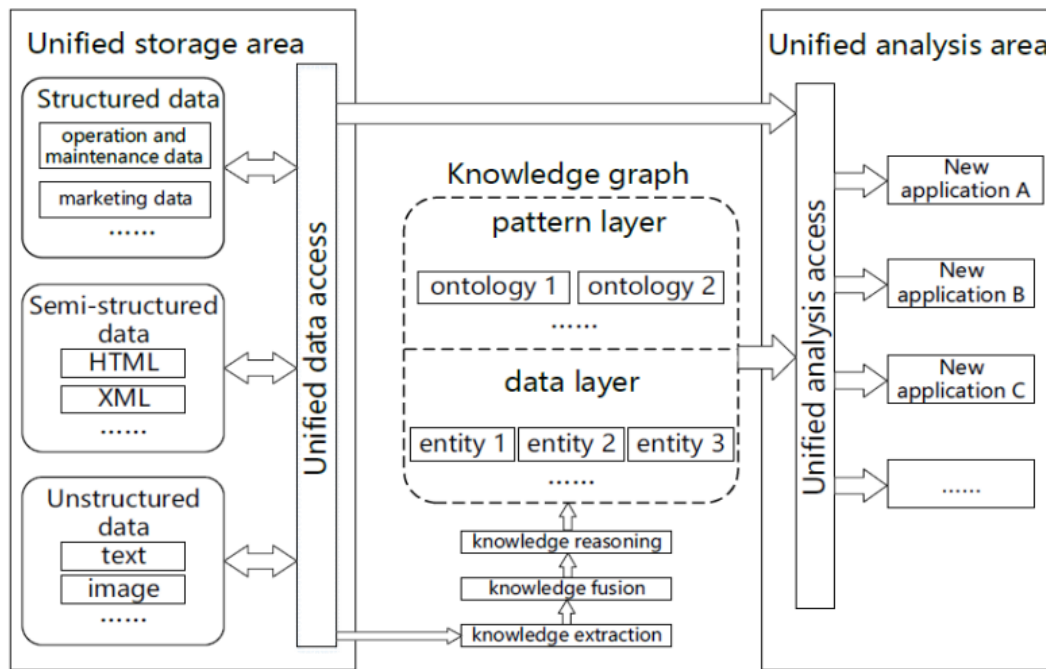
Dengan demikian, penerapan RAG pada dokumentasi Kubernetes tidak hanya berfungsi sebagai mesin pencarian semantik, tetapi juga menjadi pendekatan mitigasi *hallucination*.

II.5 Knowledge Graph dan GraphRAG

II.5.1 Definisi dan Struktur

Knowledge Graph (KG) merupakan representasi pengetahuan dalam bentuk graf yang tersusun dari entitas (*node*) dan relasi (*edge*) yang memiliki makna semantik

(Hofer dkk. 2024). Berbeda dengan basis data konvensional yang mengandalkan skema statis, KG bersifat *schema-flexible* sehingga mampu mengintegrasikan data heterogen dari sumber terstruktur, semi-terstruktur, maupun tidak terstruktur seperti yang ditunjukkan pada Gambar II.2. Struktur KG umumnya dinyatakan melalui *triples* berbentuk (subjek–predikat–objek).



Gambar II.2 Arsitektur *Knowledge Graph* (Yu dkk. 2021)

Menurut Yu dkk. (2021), terdapat dua pendekatan utama dalam pembangunan KG, yaitu *top-down* dan *bottom-up* yang dibedakan berdasarkan urutan perancangan model dan pengisian fakta.

1. *Top-down Construction*

Pada pendekatan ini, proses pembangunan KG dimulai dari definisi model semantik terlebih dahulu. Langkah pertama mencakup perancangan *ontology library* yang memuat kelas entitas, relasi, hierarki, dan aturan semantik. Setelah struktur konseptual ditetapkan, barulah fakta atau data dimasukkan ke dalam basis pengetahuan. Dengan demikian, pendekatan *top-down* mengikuti urutan *model layer* kemudian *data layer*. Pendekatan ini umumnya digunakan pada domain yang membutuhkan konsistensi semantik tinggi seperti medis, keuangan, atau layanan publik.

2. *Bottom-up Construction*

Berbeda dari pendekatan sebelumnya, *bottom-up* mengawali proses konstruksi melalui ekstraksi data langsung dari korpus teks atau sumber data lain. De-

ngan menganalisis frekuensi entitas, dan pola semantik kemudian dibentuk secara bertahap. Artinya, *data layer* dibangun terlebih dahulu, kemudian dilanjutkan dengan penyusunan *pattern layer*. Pendekatan ini didorong oleh ketersediaan data dan sesuai untuk domain yang cepat berubah atau belum memiliki referensi ontologi.

Menurut Hofer dkk. (2024), konstruksi *knowledge graph* (KG) modern umumnya dilakukan secara semi-otomatis dan bersifat inkremental. Proses pembangunan KG mencakup sejumlah tugas inti yang dapat dijalankan secara paralel, asinkron, maupun selektif sesuai jenis sumber data dan kebutuhan penggunaan. Tugas-tugas utama tersebut adalah sebagai berikut,

1. **Manajemen Metadata**

Pengelolaan metadata terkait, seperti *provenance* entitas, informasi temporal, struktur data, kualitas data, hingga log proses. Tugas ini bersifat menyeluruh karena diperlukan pada setiap tahapan konstruksi KG.

2. ***Data Acquisition dan Preprocessing***

Pemilihan sumber data yang relevan, akuisisi data, transformasi format, serta pembersihan awal. Tingkat detail proses ini bergantung pada keragaman sumber yang digunakan.

3. **Manajemen Ontologi**

Pembuatan dan pengembangan ontologi secara bertahap untuk mendefinisikan kelas, properti, dan relasi yang membentuk struktur semantik KG.

4. ***Knowledge Extraction (KE)***

Ekstraksi informasi terstruktur dari data tidak terstruktur atau semi-terstruktur menggunakan teknik seperti *Named Entity Recognition*, *entity linking*, dan *relation extraction*.

5. ***Entity Resolution (ER) dan Fusion***

Penyatuan entitas yang merepresentasikan objek yang sama untuk menghindari duplikasi.

6. ***Knowledge Completion***

Pelengkapan KG dengan pengetahuan baru melalui inferensi, prediksi relasi hilang, atau penambahan tipe entitas untuk meningkatkan kelengkapan semantik.

7. ***Quality Assurance (QA)***

Identifikasi masalah kualitas data pada KG serta penerapan strategi perbaikan yang dapat mencakup *validation checks*, konsistensi semantik, atau pembersihan lanjutan.

Proses ini tidak selalu bersifat linear karena urutan eksekusi bergantung pada jenis

sumber data dan beberapa langkah dapat menghilangkan kebutuhan langkah lain, seperti *entity linking* yang dapat menggantikan *entity resolution*. Selain itu, *metadata management* bersifat *cross-cutting* karena berlangsung pada seluruh tahap konstruksi KG.

II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi

Sistem menggabungkan *Knowledge Graph* (KG) dengan alur RAG yang bertujuan untuk meningkatkan akurasi pencarian, kemampuan *multi-hop reasoning*, serta menjaga keterkaitan struktur dokumen. Arsitektur keseluruhan mengikuti tiga fase utama, yaitu *Indexing*, *Retrieval*, dan *Generation* (Knollmeyer, Caymazer, dan Grossmann 2025). Model arsitektur ditunjukkan pada Gambar II.3.

Arsitektur ini bekerja dalam tiga fase utama. Pada tahap *Indexing*, dokumen diekstrak dan dibentuk menjadi dua lapisan graf: *Document KG* (DKG) yang memetakan hierarki dokumen (bab–subbab–*chunk*) dan *Information KG* (IKG) yang menghubungkan antar-*chunk* melalui *keywords*; seluruh URI *chunk* juga disimpan pada basis data vektor untuk pencarian hibrid. Pada tahap *Retrieval*, sistem memperluas hasil *dense embedding* awal melalui tiga strategi graf: *Informed Chapter Search* (ICS, konteks satu bab), *Informed Keyword Search* (IKS, konteks berbasis IKG), dan *Uninformed Keyword Search* (UKS, berbasis kata kunci kueri). Pada tahap *Generation*, sistem menerapkan *pass condition* untuk membatasi konteks yang dikirim ke LLM, memastikan jawaban berbasis bukti dari konteks yang tersedia dengan tingkat *faithfulness* yang tinggi dan risiko *hallucination* yang minimal (Knollmeyer, Caymazer, dan Grossmann 2025).

II.5.3 GraphRAG: Integrasi *Knowledge Graph* dan *Large Language Models*

Large Language Models (LLM) dan *Knowledge Graphs* (KG) merupakan dua paradigma representasi pengetahuan yang saling melengkapi. LLM unggul dalam pemrosesan bahasa alami serta generalisasi lintas tugas tetapi rentan terhadap halusinasi fakta. Sebaliknya, KG menawarkan representasi pengetahuan faktual dalam bentuk struktur graf eksplisit yang kuat untuk penalaran simbolik namun kurang mampu menangani pemahaman bahasa alami yang kompleks (Pan dkk. 2024).

Untuk mengatasi kelemahan LLM tersebut, pendekatan *Retrieval-Augmented Generation* (RAG) konvensional sering digunakan. Namun, RAG tradisional bekerja dengan cara memecah dokumen menjadi potongan teks (*chunks*) independen sehingga sering kali menghilangkan informasi struktural dan relasi esensial antar-entitas (Ma dkk. 2025). Sebagai solusi komprehensif atas keterbatasan tersebut, muncullah konsep *Graph Retrieval-Augmented Generation* (GraphRAG).

Berbeda dengan pencarian semantik biasa berbasis vektor teks, GraphRAG memanfaatkan mekanisme penelusuran graf (*graph traversal*) untuk mengambil konteks terstruktur secara utuh sebelum diberikan ke LLM (Han dkk. 2024). Mekanisme ini secara langsung mengekstraksi pengetahuan relevan dari data graf dengan merangkai subgraf atau jalur relasional berdasarkan kueri pengguna. Proses ini mencakup ekstraksi subgraf, penyaringan jalur (*path-filtering*), dan penyempurnaan jalur (*path-refinement*) guna menyajikan konteks penalaran yang akurat (Ma dkk. 2025).

Secara formal, integrasi generatif ini dimodelkan sebagai fungsi yang bersyarat terhadap pengetahuan graf:

$$p_{\theta}(y \mid x, \mathcal{K}) = \sum_{z \in \mathcal{Z}(x, \mathcal{K})} p_{\theta}(y \mid x, z) p(z \mid x, \mathcal{K}), \quad (\text{II.1})$$

dengan $\mathcal{K} = \{(h, r, t) \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}\}$ merepresentasikan KG sebagai himpunan *triples*, dan $\mathcal{Z}(x, \mathcal{K})$ merupakan himpunan subgraf atau fakta terstruktur yang ditarik dari graf berdasarkan kueri x . Dalam skema ini, KG bertindak sebagai ruang pengetahuan non-parametrik penyedia panduan penalaran (*reasoning guidelines*), sedangkan LLM bertindak sebagai generator parametrik (Ma dkk. 2025).

Pendekatan GraphRAG ini menjadi sangat krusial dalam domain berskala kompleks seperti manajemen konfigurasi Kubernetes. Dengan memetakan hierarki objek dan referensi silang ke dalam graf, algoritma *traversal* dapat merangkai jejak penalaran (*reasoning path*) yang kohesif. Injeksi konteks terstruktur ini ke dalam perintah LLM terbukti secara efektif menekan tingkat halusinasi, memastikan akurasi faktual, dan mempertahankan validitas sintaksis pada hasil akhir manifes YAML yang dibangun.

II.6 Metrik Evaluasi Sistem Informasi Retrieval

Evaluasi sistem GraphRAG membutuhkan landasan metrik yang bersumber dari literatur *Information Retrieval* (IR) dan NLP. Bagian ini membahas konsep metrik standar yang menjadi acuan, sedangkan metrik khusus domain Kubernetes (seperti *Graph Coverage*, *Edge Coverage*, *Hop-Accuracy*, *Scope Accuracy*, dan *Graph Field Compliance*) dirancang sebagai kontribusi penelitian dan diperkenalkan pada Bab III.

II.6.1 Metrik Kualitas Jawaban

Evaluasi kualitas jawaban sistem RAG dalam literatur umumnya menggunakan dua metrik berikut:

1. *Faithfulness*

Mengukur sejauh mana jawaban yang dihasilkan oleh LLM bersumber dari konteks yang di-*retrieve*, bukan dari pengetahuan parametrisnya sendiri (Es dkk. 2023). Metrik ini secara langsung mengukur risiko halusinasi: jawaban yang tinggi *faithfulness*-nya berarti setiap klaim dapat ditelusuri ke konteks yang disediakan.

$$\text{Faithfulness} = \frac{|\text{klaim berbasis konteks}|}{|\text{total klaim}|} \quad (\text{II.2})$$

2. *Answer Relevance*

Mengukur kesesuaian semantik antara jawaban yang dihasilkan dan maksud asli kueri pengguna (Es dkk. 2023). Implementasinya umumnya berbasis kemiripan kosinus antara representasi vektor jawaban dan pertanyaan.

II.6.2 Metrik Kualitas Retrieval

Evaluasi komponen *retrieval* menggunakan metrik standar IR yang mengukur ketepatan dan kelengkapan dokumen yang diambil:

1. *Precision@k* dan *Recall@k*

Precision@k mengukur proporsi dokumen relevan di antara k dokumen teratas yang diambil, sedangkan *Recall@k* mengukur proporsi dokumen relevan yang berhasil ditemukan dari seluruh dokumen relevan yang ada (Zhao dkk. 2024).

2. *F1-Score@k*

Merupakan rata-rata harmonik antara *Precision@k* dan *Recall@k*, memberikan gambaran seimbang tentang trade-off antara ketepatan dan kelengkapan (Zhao dkk. 2024):

$$\text{F1@k} = 2 \times \frac{\text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}} \quad (\text{II.3})$$

3. *NDCG@k (Normalized Discounted Cumulative Gain)*

Mengevaluasi kualitas peringkat hasil *retrieval* dengan memberikan bobot lebih tinggi pada dokumen relevan yang muncul di posisi atas (Es dkk. 2023):

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}} \quad (\text{II.4})$$

Metrik ini penting dalam konteks RAG karena dokumen relevan yang terpotong akibat batasan token LLM tidak memberikan manfaat apapun.

II.6.3 Metrik Validitas Sintaksis YAML

Untuk evaluasi keluaran konfigurasi YAML, dua metrik standar yang relevan adalah:

1. *Syntactic Validity*

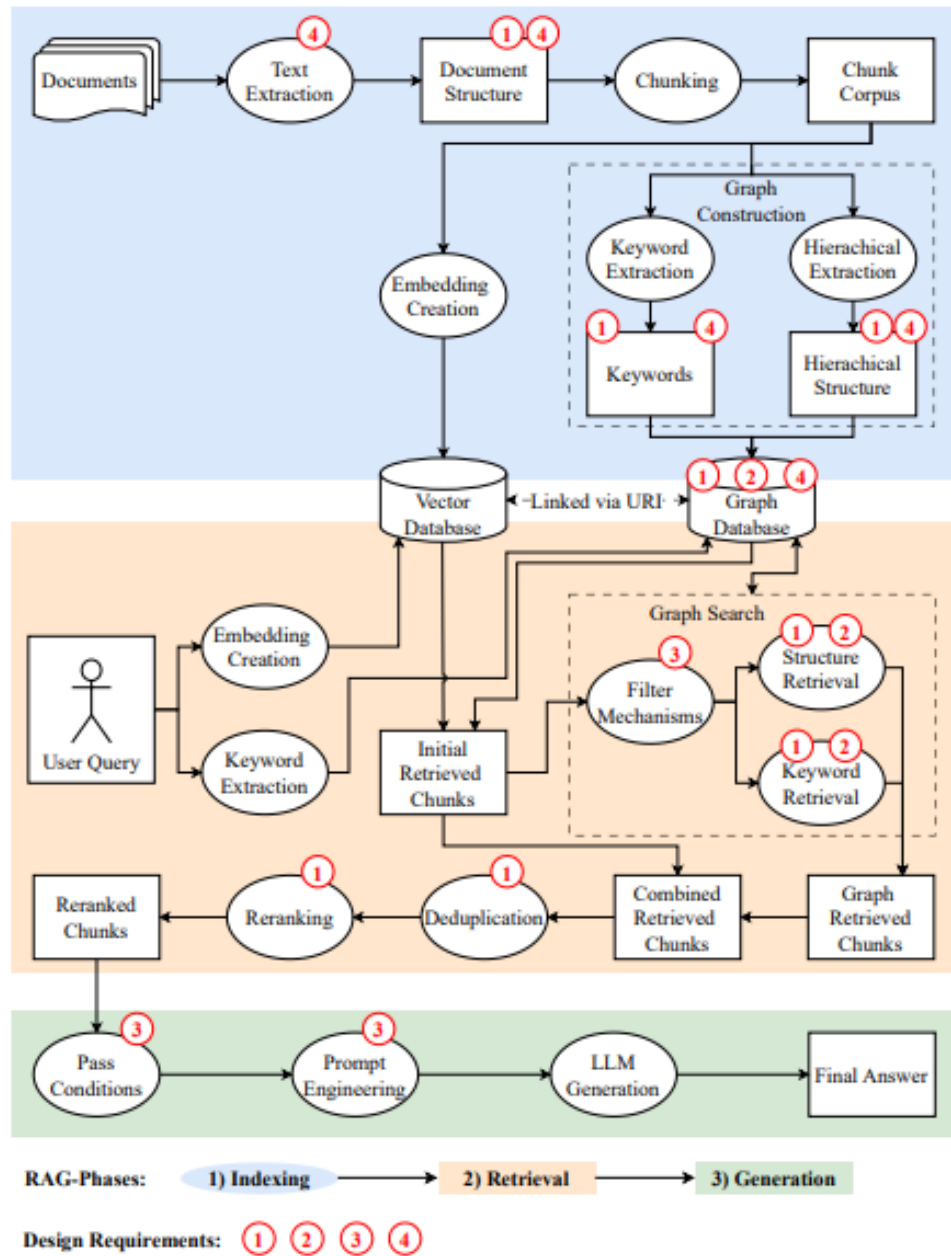
Mengukur persentase keluaran YAML yang dapat diurai tanpa galat menggunakan *parser* standar seperti `pyyaml`. Metrik ini bersifat biner: dokumen YAML valid atau tidak valid.

2. *Schema Compliance*

Mengukur kelengkapan *required fields* berdasarkan spesifikasi skema yang berlaku, dalam konteks ini spesifikasi OpenAPI Kubernetes v1.30 (The Kubernetes Authors 2024b). Catatan: kedua metrik ini hanya relevan untuk *fixture* bertipe `yaml_gen`.

Kerangka evaluasi tiga dimensi (*Answer Quality*, *Retrieval Quality*, *Reasoning Quality*) beserta metrik khusus yang dikembangkan untuk domain Kubernetes diuraikan secara rinci pada Bab III.

Berdasarkan kajian literatur yang telah dipaparkan, teridentifikasi dua kesenjangan utama yang menjadi landasan penelitian ini. Pertama, pendekatan RAG berbasis vektor konvensional terbukti tidak memadai untuk domain Kubernetes karena gagal menangkap relasi hierarkis dan referensial yang menentukan validitas konfigurasi (Wan dkk. 2025). Kedua, meskipun integrasi *Knowledge Graph* dalam RAG telah menunjukkan peningkatan signifikan pada domain umum, belum ada rancangan taksonomi relasi dan mekanisme *traversal* yang secara spesifik disesuaikan untuk skema OpenAPI Kubernetes. Bab selanjutnya menganalisis karakteristik data, kebutuhan sistem, dan alternatif pendekatan solusi untuk menjawab kedua kesenjangan tersebut.



Gambar II.3 Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Bagian ini menganalisis permasalahan yang muncul dalam proses pengembangan sistem *cloud-native* berbasis Kubernetes, ditinjau dari dua sudut utama yaitu pemahaman masalah bisnis (*Business Understanding*) dan kondisi data yang menjadi sumber permasalahan (*Data Understanding*).

III.1.1 Pemahaman Masalah Bisnis (*Business Understanding*)

Transformasi digital di berbagai industri mendorong adopsi arsitektur *cloud-native* berbasis *microservices* yang dikelola melalui Kubernetes. Dalam praktiknya, Kubernetes tidak hanya berfungsi sebagai alat operasional, tetapi juga sebagai aset strategis yang menentukan kecepatan dan stabilitas siklus pengembangan perangkat lunak (*software development lifecycle*). Kompleksitas konfigurasi yang tinggi, kurva pembelajaran yang curam, serta kebutuhan untuk melakukan *onboarding* bagi *engineer* baru menjadikan Kubernetes sebuah titik krusial dalam kapasitas operasional.

Permasalahan terkait pengelolaan konfigurasi Kubernetes tidak hanya bersifat teknis, tetapi menimbulkan dampak bisnis berupa biaya operasional dan risiko produksi. Beberapa permasalahan utama yang diidentifikasi adalah sebagai berikut:

- 1. B01 - Inefisiensi Waktu (*Operational Cost*)**

Dokumentasi Kubernetes bersifat luas dan terfragmentasi sehingga *developer/DevOps* membutuhkan waktu yang cukup lama untuk menelusuri relasi antar-objek Kubernetes. Waktu pencarian informasi yang berlebih menjadi pemborosan sumber daya, terutama pada posisi *engineer* dengan biaya tenaga kerja tinggi.

- 2. B02 - Kesenjangan Pengetahuan (*Knowledge Gap*)**

Dokumentasi statis tidak sepenuhnya mendukung cara berpikir asosiatif manusia ketika memahami struktur spesifikasi. Sistem generatif berbasis LLM pun masih sering menghasilkan *hallucinated fields* karena tidak memiliki pemahaman skematik. Hal ini bisa memperburuk risiko kesalahan konfigurasi.

3. B03 - Risiko Kesalahan Konfigurasi (*Operational Risk*)

Kesalahan sintaksis atau penempatan atribut berkas konfigurasi YAML yang tidak sesuai spesifikasi dapat menyebabkan *deployment failure* hingga *downtime*. *Downtime* produksi berimplikasi langsung pada kerugian finansial, reputasi layanan, dan penundaan proses bisnis.

Ketiga butir *business understanding* tersebut menjadi dasar dalam penyusunan rumusan masalah penelitian. B01 mengenai inefisiensi waktu akibat dokumentasi yang terfragmentasi mengarah pada kebutuhan representasi pengetahuan yang lebih terstruktur sehingga dirumuskan sebagai Rumusan Masalah 1 tentang rekonstruksi spesifikasi OpenAPI menjadi *knowledge graph*. B02 yang menyoroti kesenjangan pemahaman skematik dan risiko *hallucination* pada LLM memunculkan Rumusan Masalah 2 terkait perancangan mekanisme *graph-guided retrieval* untuk menghadirkan konteks struktural yang akurat. Sementara itu, B03 terkait risiko kesalahan konfigurasi dan dampak operasionalnya diterjemahkan menjadi Rumusan Masalah 3 yang berfokus pada evaluasi efektivitas GraphRAG dalam meningkatkan relevansi konteks, *faithfulness*, serta validitas sintaksis dibandingkan pendekatan *baseline*.

III.1.2 Pemahaman Data (*Data Understanding*)

Bagian ini menjelaskan karakteristik data dokumentasi API Kubernetes yang menjadi sumber utama dalam proses konstruksi *Graph-based Retrieval Augmentation*. Analisis mencakup identitas data, struktur, pola hubungan antar-objek Kubernetes, keterbatasan kualitas, serta seleksi data relevan yang digunakan dalam penelitian.

III.1.2.1 Sumber dan Spesifikasi Data (*Data Source & Specifications*)

Data penelitian diperoleh dari repositori resmi Kubernetes pada GitHub dengan alamat `kubernetes/kubernetes`. Data diambil dalam bentuk berkas `swagger.json` yang mengikuti standar *OpenAPI Specification* (OAS). Penelitian ini menggunakan versi Kubernetes **v1.30** sebagai acuan. Berkas `swagger.json` memiliki ukuran rata-rata 3,757 MB dan memuat 2.500–3.500 definisi API. Kompleksitas tersebut menjustifikasi penggunaan metode ekstraksi otomatis, karena pendekatan manual berpotensi menghasilkan kesalahan interpretasi struktur dan kehilangan relasi semantik antar-objek Kubernetes.

III.1.2.2 Analisis Struktur Dokumen

Berdasarkan analisis terhadap struktur `swagger.json`, terdapat dua komponen utama beserta karakteristik hierarkisnya:

1. **Definitions (atau Schema Resources):** Mendeskripsikan spesifikasi objek Kubernetes seperti *Pod*, *Service*, dan *Deployment*. Hampir seluruh objek Ku-

bernetes mematuhi pola struktural wajib yang terdiri atas *apiVersion*, *kind*, *metadata*, *spec*, dan *status*. Namun, isi dari atribut *spec* sangat bervariasi antar-komponen di dalam teks YAML. Akibatnya, waktu pembuatan konfigurasi meningkat dan pengembang sangat bergantung pada panduan eksternal. Secara spesifik, setiap definisi skema di bagian ini memuat:

- a. *properties*: Merepresentasikan ruas (*field*) aktual pada berkas konfigurasi YAML.
 - b. *type*: Menentukan format tipe data (seperti *object*, *array*, *string*, *integer*, *boolean*).
 - c. *required*: Mendefinisikan daftar ruas konfigurasi wajib isi.
 - d. *description*: Menyediakan dokumentasi tekstual mengenai fungsionalitas ruas.
 - e. *\$ref*: Bertindak sebagai referensi silang ke definisi skema bersarang (*nested schema*). Ciri utama dari struktur dokumen ini adalah definisi yang saling mereferensikan secara mendalam. Akibatnya, satu ruas YAML dapat mengarah ke berbagai kemungkinan bentuk, meliputi:
 - i. Definisi objek skema lain.
 - ii. Struktur daftar (*list*).
 - iii. Struktur pemetaan (*map*) dengan batasan (*constraint*) khusus.
2. ***Paths***: Mendeskripsikan titik akhir (*endpoint*) API, misalnya `GET /api/v1/pods`, sebagai antarmuka interaksi objek Kubernetes dalam ekosistem kluster. Setiap titik akhir memiliki atribut penentu interaksi komunikasi yang memuat:
- a. Metode HTTP fungsional (*GET*, *POST*, *PUT*, *PATCH*, *DELETE*).
 - b. Parameter injeksi pada jalur (*path*) dan kueri (*query*).
 - c. Skema balasan dari peladen (*response schema*).
 - d. Relasi struktural pembentuk fungsionalitas yang mengarah kembali ke blok *definitions*.

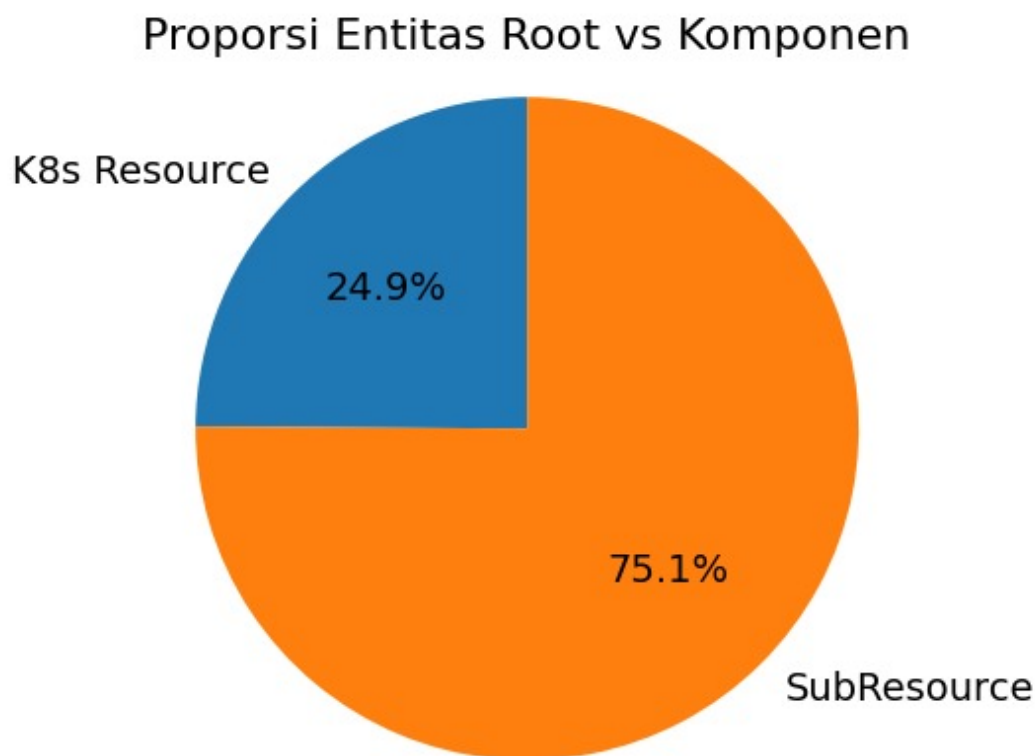
III.1.2.3 Analisis Data Eksploratif Spesifikasi OpenAPI

Sebelum merancang skema *knowledge graph*, analisis data eksploratif (*Exploratory Data Analysis* / EDA) dilakukan terhadap berkas `kubernetes_swagger.json` berukuran 3,67 MB guna memetakan skala, kompleksitas struktural, dan sebaran definisi skema. Proses ekstraksi awal berhasil mengidentifikasi 735 definisi skema. Sebanyak 5 entri diabaikan karena teridentifikasi sebagai data bising (*noise*), menyisakan 730 definisi skema valid untuk dievaluasi lebih lanjut.

Temuan utama dari proses EDA ini dikelompokkan ke dalam tiga metrik krusial:

1. **Distribusi objek Kubernetes *root* vs komponen pendukung**

Klasifikasi definisi skema menunjukkan ketimpangan proporsi yang sangat signifikan antara objek utama dan objek pendukung. Dari total 730 definisi, hanya 183 (24,9%) terklasifikasi sebagai objek Kubernetes akar (*root resources*). Objek akar ini merupakan objek mandiri penyangga atribut *GroupVersionKind* (GVK) seperti *Deployment* atau *Pod* sehingga dapat diterapkan secara langsung ke dalam kluster. Sementara itu, mayoritas mutlak sebanyak 547 definisi (75,1%) hanyalah skema pendukung (*sub-resources*) pembangun komponen akar tersebut. Ketimpangan ini membuktikan betapa masifnya dependensi struktural di dalam Kubernetes.



Gambar III.1 Proporsi objek Kubernetes *root* vs komponen pendukung dalam spesifikasi OpenAPI Kubernetes

2. Kepadatan Properti dan Intensitas Referensi Silang

Inspeksi terhadap struktur properti mengungkap bahwa setiap definisi skema rata-rata memuat 4,1 ruas (*field*), dengan objek Kubernetes paling kompleks menampung hingga 44 ruas parameter. Temuan paling krusial yang memvalidasi tingginya kompleksitas hierarki adalah keberadaan referensi silang (*\$ref*). Sebanyak 437 definisi (hampir 60% dari total data) terbukti mengandalkan referensi ke skema lain dengan rata-rata 1,11 referensi per definisi. Tingginya intensitas referensi ini secara kuantitatif memvalidasi argumen

mengenai sulitnya melacak skema bersarang (*nested schema*) secara manual.

Keseluruhan temuan kuantitatif EDA ini secara langsung menjadi landasan arsitektural dalam perancangan skema graf. Pemetaan atas 149 jenis (*kind*) objek Kubernetes unik hasil analisis ini secara presisi diproyeksikan akan membentuk sebuah *knowledge graph* Kubernetes yang memuat 730 *node* pendefinisi objek beserta estimasi 1.486 *edge* pembentuk topologi infrastrukturnya.

III.1.2.4 Analisis Keterhubungan Data

1. Relasi Tingkat Skema (*Schema-Level*) – Tercakup Penuh 100%

Kategori ini terpetakan dan direpresentasikan secara utuh di dalam graf karena bersumber langsung dari struktur statis spesifikasi OpenAPI. Relasi pada tingkat ini mencakup:

- a. **Sifat Atomik Pod (*Passive Relationships*):** Pod bertindak sebagai unit eksekusi atomik pasif tanpa relasi skema eksplisit ke luar. Relasinya terbentuk secara pasif akibat interaksi struktural dari objek Kubernetes eksternal.
- b. **Komposisi Hierarkis (*Hierarchical Composition*):** Terjadi ketika satu objek Kubernetes mengandung definisi objek lain secara utuh. Contoh utamanya adalah Deployment mereferensikan Pod melalui blok `spec.template` penyimpanan keseluruhan spesifikasi PodSpec.
- c. **Referensi Langsung (*Direct Reference & Injection*):** Terjadi ketika sebuah objek Kubernetes memanggil objek lain secara eksplisit pada parameternya. Contohnya adalah injeksi konfigurasi satu arah dari ConfigMap atau Secret ke lingkungan eksekusi Pod seperti pada Kode III.1 dan

1	<code>env :</code>	1	<code>env :</code>
2	<code>valueFrom :</code>	2	<code>valueFrom :</code>
3	<code>configMapKeyRef :</code>	3	<code>secretKeyRef :</code>
4	<code>name: "app-config"</code>	4	<code>name: "db-secret"</code>
5		5	

Listing III.1 Injeksi ConfigMap

Listing III.2 Injeksi Secret

2. Relasi Semi-Statis (*Semi-Static*) – Tercakup Parsial

Kategori ini terpetakan ke dalam graf secara konseptual atau sebagai properti *node*, namun tidak membentuk jalur semantik fungsional penuh.

- a. **Asosiasi Label/Selektor (*Label/Selector Association*):** Direpresentasikan sekadar sebagai *edge* konseptual pembentuk topologi graf.
- b. **Deteksi Cakupan Klaster (*Cluster-Scope Detection*):** Pembatasan area operasional tidak dipetakan sebagai ruang graf terpisah melainkan dia-

komodasi melalui atribut properti cakupan (*scope property*) pada setiap *node*.

- c. **Anotasi dan Toleransi (*Annotations/Tolerations*):** Nilai konfigurasi tambahan ini hanya disimpan sebagai properti pada *node* tanpa memicu pembentukan *edge* semantik dengan objek Kubernetes lain.

3. Relasi Tingkat Operasional (*Runtime-Level*) – Tidak Tercakup

Kategori ini secara mutlak berada di luar cakupan fungsional sistem GraphRAG.

Relasi pada tingkat ini mencakup:

- a. **Hierarki Kepemilikan (*OwnerReferences*):** Merupakan atribut meta-data operasional penentu pengumpulan sampah sumber daya (*garbage collection*) saat infrastruktur berjalan.
- b. **Interaksi Lintas Ruang Nama (*Cross-Namespace Interactions*):** Melibatkan perilaku pembatas akses operasional jaringan secara langsung (*runtime behavior*).
- c. **Logika Pencocokan Label (*Label Matching Logic*):** Merupakan proses komputasi evaluasi dinamis oleh layanan kluster untuk menentukan target pembagian beban jaringan sesungguhnya.

4. Keterbatasan Inheren dan Pengembangan Lanjutan

Ketiadaan pemetaan pada relasi tingkat operasional merupakan batasan inheren dalam penelitian ini. Hal ini terjadi karena arsitektur GraphRAG dibangun secara eksklusif bermodalkan skema OpenAPI (*swagger.json*) penyedia representasi struktur statis dan bukan pembaca status kluster yang sedang beroperasi (*runtime state*). Sebagai langkah pengembangan lanjutan (*future work*) guna mengakomodasi kasus penggunaan pelacak relasi dinamis, sistem pemetaan graf ini dapat diintegrasikan secara langsung dengan mekanisme pemantauan kontroler Kubernetes (*controller watch mechanism*) atau penerapan pola operator (*operator pattern*).

III.1.2.5 Desain Taksonomi *Edge* untuk *Knowledge Graph* Kubernetes

Berdasarkan analisis pola keterhubungan data di atas, penelitian ini merancang 18 jenis *edge* yang terdistribusi dalam 7 kategori relasi. Taksonomi ini adalah **kontribusi penelitian** yang dirancang khusus berdasarkan semantik domain Kubernetes, bukan standar yang ada di literatur sebelumnya. Seluruh 18 *edge type* diimplementasikan dalam kode pada variabel `_ALL_EDGE_TYPES` di `src/graph/queries.py`.

Tabel III.1 Taksonomi 18 jenis *edge* dalam *knowledge graph* Kubernetes

No	<i>Edge Type</i>	Kategori	Contoh Relasi
1	HAS_PROPERTY	Struktural	<i>Deployment</i> → <i>spec</i>
2	EXTENDS	Struktural	<i>Deployment</i> → <i>ObjectMeta</i>
3	ONE_OF	Struktural	<i>Volume</i> → satu sumber penyimpanan
4	ANY_OF	Struktural	<i>EnvFromSource</i> → <i>ConfigMap/Secret</i>
5	CONTAINS_POD_TEMPLATE	<i>Workload</i>	<i>Deployment</i> → <i>PodTemplateSpec</i>
6	CONTAINS_JOB_TEMPLATE	<i>Workload</i>	<i>CronJob</i> → <i>JobTemplateSpec</i>
7	HAS_CONTAINER	<i>Workload</i>	<i>PodSpec</i> → <i>Container</i>
8	CLAIMS_VOLUME	Penyimpanan	<i>StatefulSet</i> → <i>PersistentVolumeClaim</i>
9	MOUNTS_VOLUME	Penyimpanan	<i>PodSpec</i> → <i>Volume</i>
10	USES_STORAGE_CLASS	Penyimpanan	<i>PVC</i> → <i>StorageClass</i>
11	LOADS_CONFIGMAP	Konfigurasi	<i>Container</i> → <i>ConfigMap</i>
12	USES_SECRET	Konfigurasi	<i>PodSpec</i> → <i>Secret</i>
13	SELECTS_POD	Jaringan	<i>Service</i> → <i>Pod</i> via <i>selector</i>
14	ROUTES_TO_SERVICE	Jaringan	<i>Ingress</i> → <i>Service</i>
15	BINDS_ROLE	RBAC	<i>RoleBinding</i> → <i>Role</i>
16	BINDS_SERVICE_ACCOUNT	RBAC	<i>RoleBinding</i> → <i>ServiceAccount</i>
17	USES_SERVICE_ACCOUNT	RBAC	<i>PodSpec</i> → <i>ServiceAccount</i>
18	SCALES_RESOURCE	<i>Autoscaling</i>	<i>HPA</i> → <i>Deployment/StatefulSet</i>

III.1.2.6 Justifikasi Batas Kedalaman Penelusuran Graf (3 Lompatan)

Batas maksimum 3 lompatan (*hops*) dalam penelusuran graf ditetapkan berdasarkan analisis distribusi kedalaman skema OpenAPI Kubernetes. Tabel III.2 menunjukkan bahwa karakteristik *node* berubah secara signifikan seiring bertambahnya kedalaman:

Tabel III.2 Distribusi karakteristik *node* berdasarkan kedalaman penelusuran graf

Kedalaman	Karakteristik <i>Node</i>	Contoh	Penggunaan
1–2	Spesifik <i>resource</i> (dibagikan 2–3 <i>resource</i>)	<i>DeploymentSpec</i> , <i>ServiceSpec</i>	Tipe <i>explain</i> , <i>followup</i>
3	<i>Field</i> tingkat kontainer	<i>image</i> , <i>ports</i> , <i>env</i> , <i>resources</i>	Tipe <i>generate_yaml</i> , <i>trace_relationship</i>
4+	Tipe generik (dibagikan 19–136 <i>resource</i>)	<i>Quantity</i> , <i>IntOrString</i> , <i>LocalObjectReference</i>	Tidak digunakan (menurunkan presisi)

Dengan menggunakan kedalaman yang disesuaikan per tipe *intent* (kedalaman 2 untuk *explain/followup*, kedalaman 3 untuk *generate_yaml/trace_relationship*), sistem mengoptimalkan *Context Precision* dengan menghindari injeksi tipe generik yang tidak informatif ke dalam konteks LLM.

III.1.2.7 Analisis Kompleksitas Pertanyaan dan Batasan Kemampuan Sistem

Batasan kedalaman traversal memiliki implikasi langsung terhadap jenis pertanyaan yang dapat dijawab secara optimal. Tabel III.3 mengklasifikasikan pertanyaan pengguna ke dalam empat tingkat berdasarkan kedalaman informasi yang dibutuhkan dan cakupan pengetahuan graf.

Tabel III.3 Klasifikasi Tingkat Kompleksitas Pertanyaan Pengguna

Tier	Kategori	Karakteristik	Kualitas
1	Optimal	Satu <i>resource</i> , informasi pada kedalaman ≤ 3 hop. Contoh: “ <i>Apa itu Deployment?</i> ”, “ <i>Buat YAML StatefulSet dengan PVC.</i> ”	Tinggi
2	Terdegradasi	Informasi pada kedalaman 4–5 hop, atau melibatkan dua <i>resource</i> utama. Contoh: “ <i>Resource limits container di dalam Deployment.</i> ”	Sedang
3	Di Luar Graf	Membutuhkan data <i>runtime</i> atau perintah operasional. Contoh: “ <i>Kenapa Pod saya CrashLoopBackOff?</i> ”	Rendah
4	Di Luar Domain	Tidak berkaitan dengan Kubernetes dan ekosistemnya.	Ditolak

Ketika pertanyaan melampaui batas Tier 1, sistem tidak mengembalikan pesan kesalahan, melainkan mengisi kesenjangan konteks menggunakan pengetahuan parametrik LLM (*silent degradation*). Kondisi ini menjadi salah satu faktor yang menjelaskan perbedaan nilai *Retrieval Quality* dan *Answer Quality* pada hasil evaluasi.

III.2 Analisis Kebutuhan

Tahap analisis kebutuhan berfokus pada identifikasi kapabilitas yang harus dimiliki sistem guna menjawab tujuan penelitian dan kebutuhan bisnis. Untuk memastikan bahwa kebutuhan yang dirumuskan bersifat relevan dengan masalah yang dihadapi, dilakukan analisis kesenjangan teknis (*technical gap*) pada pendekatan yang ada saat ini pada sistem *Retrieval-Augmented Generation* pada Tabel III.4.

Tabel III.4 *Technical Gap Analysis* pada Domain Kubernetes

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T01 - Representasi Struktur Data	Menggunakan pemecahan teks secara <i>linear chunking</i> , yaitu dokumen hierarkis dipecah menjadi potongan terpisah sehingga hubungan <i>parent-child</i> terputus (Cao dkk. 2025).	Terjadi fenomena <i>loss of deep context</i> . Sistem gagal memahami kedalaman hierarki, misalnya properti <i>limits</i> merupakan turunan bersarang dari Deployment, karena strukturnya terpisah antar <i>chunk</i> (Kotstein dan Decker 2024).
T02 - Mekanisme Validasi	Validasi bergantung pada model LLM atau aturan manual berbasis <i>hard constraints</i> yang harus ditulis satu per satu (Wan dkk. 2025).	Risiko terjadinya <i>syntactic hallucination</i> tinggi (LLM mengarang <i>field</i>), atau beban pemeliharaan menjadi besar saat aturan manual harus diperbarui (Gao dkk. 2024).
T03 - Ambiguitas Entitas	Mengandalkan frekuensi kemunculan kata kunci (<i>keyword frequency</i>) atau kedekatan vektor semata.	Terdapat ambiguitas pada pola <i>loose coupling</i> karena <i>keywords</i> (misal: <i>labels</i> , <i>ports</i>) muncul berulang di banyak objek berbeda. Hal ini menyebabkan tercampurnya konteks antar-objek (<i>contextual mixing</i>).
T04 - Pemeliharaan Pengetahuan	Membutuhkan <i>fine-tuning</i> ulang model atau pembaruan aturan manual ketika terjadi perubahan versi API (Gao dkk. 2024).	Tidak efisien, mudah menjadi <i>obsolete</i> , dan sensitif terhadap perubahan versi Kubernetes yang dirilis secara berkala (± 4 bulan).

bersambung ke halaman berikutnya

Tabel III.4 *Technical Gap Analysis* pada Domain Kubernetes (lanjutan)

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T05 - Akurasi Jawaban	Sistem cenderung menghasilkan jawaban naratif umum dan tidak memberikan jaminan presisi sintaksis konfigurasi YAML.	Tidak mampu menghasilkan artefak konfigurasi yang sesuai karena sering ditemukan kesalahan format atau struktur.

III.2.1 Kebutuhan Fungsional

Berdasarkan *Business Understanding* (B01–B03), diidentifikasi sejumlah urgensi bisnis yang dianalisis lebih lanjut hingga menghasilkan beberapa celah teknis (*technical gaps*) yang menjadi dasar perumusan kebutuhan sistem. Setiap *technical gap* kemudian dijawab melalui perancangan kebutuhan fungsional yang akan diimplementasikan untuk menyelesaikan permasalahan bisnis yang ada. Tabel III.5 berikut memetakan hubungan antara kebutuhan bisnis, *technical gap*, serta kebutuhan fungsional yang dirumuskan untuk mengatasinya.

Tabel III.5 Pemetaan Kebutuhan Bisnis terhadap *Technical Gap* dan Kebutuhan Fungsional

Business Requirement (B)	Technical GAP (T)	Functional Requirement (F)
B01 – Inefisiensi waktu	T01 – Representasi struktur data	F-01 <i>Structured Schema Representation</i>
		F-02 <i>Structured Relation Retrieval</i>
	T03 – Ambiguitas entitas	F-03 <i>Intent Classification</i>
		F-04 <i>Context Construction & Injection</i>
B02 – Kesenjangan pengetahuan	T02 – Mekanisme validasi	F-05 <i>Attribute Enrichment</i>
		F-06 <i>CLI Interaction Interface</i>
	T04 – Pemeliharaan pengetahuan	F-07 <i>Selective Components Parsing</i>
B03 – Risiko kesalahan konfigurasi	T05 – Akurasi jawaban	F-08 <i>Conditional YAML Generation Constraint</i>

Berdasarkan hasil pemetaan pada Tabel III.5, berikut adalah kebutuhan fungsional yang dibutuhkan oleh sistem,

Tabel III.6 Kebutuhan Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
F-01	<i>Structured Object Representation</i>	Sistem harus mengubah skema objek ke dalam suatu representasi terstruktur yang membedakan setiap komponen, <i>field</i> , dan relasi antar-objek sehingga dapat diakses kembali secara terprogram pada tahap <i>retrieval</i> .

bersambung ke halaman berikutnya

Tabel III.6 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-02	<i>Structured Relation Retrieval</i>	Sistem harus menyediakan mekanisme <i>retrieval</i> yang mampu mengambil himpunan objek dan <i>field</i> yang saling berkaitan (misalnya hubungan antara <i>Deployment</i> , <i>Pod</i> , dan <i>Service</i>) berdasarkan entitas awal dan <i>intent</i> pengguna sebagaimana didefinisikan pada F-03.
F-03	<i>Intent Classification</i>	Sistem harus mampu (1) mendeteksi entitas Kubernetes yang dirujuk pengguna (misalnya <i>Service</i> , <i>Pod</i>), dan (2) mengklasifikasikan maksud pengguna ke dalam empat tipe <i>intent</i> : dua tipe utama, yaitu explain (pertanyaan definisi, cara kerja, atau tujuan suatu <i>resource</i>) dan generate_yaml (permintaan pembuatan konfigurasi YAML), serta dua mode pendukung, yaitu trace_relationship (penelusuran relasi antar- <i>resource</i>) dan followup (modifikasi atau perluasan jawaban sebelumnya). Setiap tipe <i>intent</i> mengontrol kedalaman penelusuran graf: <i>explain/followup</i> menggunakan kedalaman 2, sedangkan <i>generate_yaml/trace_relationship</i> menggunakan kedalaman 3.
F-04	<i>Context Construction & Injection</i>	Sistem harus mengubah hasil F-02 menjadi sebuah konteks dan menyertakannya sebagai bagian dari <i>prompt</i> yang dikirimkan ke LLM.

bersambung ke halaman berikutnya

Tabel III.6 Kebutuhan Fungsional Sistem (lanjutan)

Kode	Parameter	Deskripsi Spesifikasi
F-05	<i>Attribute Enrichment</i>	Sistem harus menyimpan metadata untuk setiap <i>field</i> konfigurasi di dalam representasi terstruktur sehingga informasi tersebut tersedia saat konteks disiapkan untuk menjawab pertanyaan pengguna.
F-06	<i>Web Interaction Interface</i>	Sistem harus menyediakan antarmuka web interaktif berbasis Streamlit yang memungkinkan pengguna memasukkan pertanyaan dalam bahasa alami, menampilkan jawaban beserta konfigurasi YAML (jika relevan), serta menyajikan jejak penalaran graf (<i>reasoning path</i>) secara visual dengan pesan kesalahan yang jelas.
F-07	<i>Conditional YAML Generation Constraint</i>	Sistem harus menginstruksikan LLM untuk menghasilkan keluaran berupa blok kode YAML yang valid secara sintaksis apabila <i>intent</i> pengguna diklasifikasikan sebagai generate_yaml . Untuk <i>intent explain</i> , trace_relationship , dan followup , sistem tidak boleh memaksakan format YAML dan harus menghasilkan jawaban naratif.

III.2.2 Kebutuhan Non Fungsional

Kebutuhan non-fungsional mendefinisikan karakteristik kualitas yang harus dimiliki oleh sistem untuk memastikan kinerja, keandalan, dan keamanan operasional. Kebutuhan ini penting untuk menjamin bahwa sistem tidak hanya berfungsi dengan benar, tetapi juga memenuhi standar kualitas yang diharapkan oleh pengguna. Berikut adalah kebutuhan non-fungsional utama yang diidentifikasi:

Tabel III.7 Kebutuhan Non-Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
NF-01 (Reliability)	<i>Fail-Safe Mechanism</i>	Sistem harus mencegah keluaran halusina- si melalui mekanisme <i>error handling</i> , ya- itu (1) Jika entitas tidak ditemukan, sis- tem harus memberi respons “Entity not found”. (2) Jika YAML terdeteksi ti- dak valid oleh validator internal, sistem harus menampilkan peringatan “Warning: Potential Syntax Error” pada antarmuka pengguna.
NF-02 (Performance)	<i>Retrieval–Generation Latency</i>	Waktu total yang dibutuhkan untuk memp- roses satu permintaan pengguna (<i>end-to- end latency</i>) tidak boleh melebihi 60 detik pada lingkungan pengujian standar.
NF-03 (Usability)	<i>Web UI Usability Stand- ard</i>	Antarmuka web interaktif berbasis Stre- amlit harus menyediakan pengalaman penggunaan yang konsisten, mudah dipa- hami, dan informatif melalui komponen <i>chat input</i> , panel jawaban, serta visualisa- si jejak penalaran graf.
NF-04 (Portability)	<i>Resource Constraints</i>	Seluruh komponen sistem harus dapat di- jalankan pada lingkungan lokal (<i>Local En- vironment</i>) dengan spesifikasi perangkat keras terbatas (minimal RAM 8GB), tan- pa ketergantungan pada infrastruktur <i>clo- ud</i> terdistribusi.

III.2.3 Pemetaan Kebutuhan Bisnis terhadap Kerangka Evaluasi

Untuk memastikan bahwa setiap dimensi evaluasi sistem memiliki dasar bisnis yang jelas, Tabel III.8 memetakan hubungan antara kebutuhan bisnis (B01–B03), dimensi evaluasi, dan metrik yang digunakan.

Tabel III.8 Pemetaan kebutuhan bisnis terhadap dimensi dan metrik evaluasi

Kebutuhan Bisnis	Pertanyaan Evaluasi	Dimensi	Metrik Utama
B01 – Inefisiensi Waktu	Apakah <i>retrieval</i> efisien dan tepat sasaran?	RetQ (35%)	F1@k, NDCG@k, Graph Coverage
B02 – Kesenjangan Pengetahuan	Apakah penalaran berbasis graf menghasilkan konteks yang benar?	ReaQ (25%)	Hop-Accuracy, Multi-Hop Success Rate, Hallucination Rate
B03 – Risiko Konfigurasi	Apakah keluaran YAML valid dan relevan terhadap kueri?	AnsQ (40%)	Syntactic Validity, Schema Compliance, Answer Relevance, Faithfulness

Pemetaan ini mempertegas bahwa dimensi evaluasi bukan sekadar metrik teknis, melainkan representasi langsung dari dampak bisnis yang ingin dimitigasi oleh sistem GraphRAG.

III.3 Alternatif Solusi

Bagian ini membahas berbagai alternatif solusi yang dapat diimplementasikan untuk memenuhi kebutuhan sistem yang telah diidentifikasi sebelumnya.

III.3.1 Hybrid KG–Vector RAG

Pendekatan *hybrid KG–Vector RAG* menggabungkan dua jalur pencarian, yakni *vector-based RAG* untuk pencarian semantik dan *knowledge graph* (KG) berbasis metadata untuk penalaran relasional (Wan dkk. 2025). Kueri pengguna diproses melalui ekstraksi entitas untuk menarik subgraf relevan dari KG sekaligus mengambil potongan teks melalui pencarian kemiripan vektor; kedua konteks terstruktur dan semantik ini kemudian digabungkan sebagai basis generasi bagi LLM.

Hasil evaluasi oleh Wan dkk. (2025) pada dataset benchmark menunjukkan peningkatan yang signifikan dibanding *vector-only RAG*: *Exact Match* meningkat dari 63,4% menjadi 77,8% (+14,4 poin persentase) dan *Context Precision* dari 69,8% menjadi 76,5% (+6,7 poin persentase). Konsekuensi dari penggabungan dua jalur

ini adalah lonjakan latensi akibat *traversal overhead* graf dan kompleksitas *prompt engineering*.

III.3.2 *GNN-augmented GraphRAG (G-Retriever)*

Arsitektur G-Retriever dirancang untuk menangani pemahaman graf tekstual berskala besar dengan memadukan *Graph Neural Networks* (GNN) dan LLM (He dkk. 2024). Sistem ini mereduksi ukuran konteks kueri melalui algoritma *Prize-Collecting Steiner Tree* (PCST) guna membangun subgraf koheren berisi *node* dan *edge* paling informatif sebelum disuntikkan ke dalam LLM.

Secara kuantitatif, pendekatan ini mampu memangkas penggunaan token hingga **99%** dibanding RAG konvensional, sekaligus menaikkan validitas *node* dari 31% menjadi **77%** (+46 poin persentase) dan validitas *edge* dari 12% menjadi **76%** tanpa memerlukan proses *fine-tuning* pada LLM (He dkk. 2024). Namun, metode PCST memiliki kompleksitas komputasi $O(|V| + |E|)$ yang semakin meningkat seiring ukuran graf, dan ketergantungan terhadap GNN menyebabkan *indexing* harus diulang setiap versi Kubernetes berubah.

III.3.3 *Hybrid Neural-Symbolic dengan Relational DB (NeuSym-RAG)*

Metode NeuSym-RAG memadukan model neural dengan basis pengetahuan simbolik berbasis aturan di dalam basis data relasional (Cao dkk. 2025). Pendekatan ini menyimpan entitas dalam tabel terstruktur dan mengeksekusi aturan simbolik untuk memvalidasi hasil inferensi LLM secara deterministik, dengan akurasi validasi mencapai lebih dari 90% pada dataset *question answering* terstruktur (Cao dkk. 2025).

Namun, skema penyimpanan relasional memiliki kelemahan struktural jika diaplikasikan pada domain Kubernetes. Dengan 730 definisi skema dan rata-rata 4,1 *field* per definisi, menelusuri relasi hierarkis bersarang (*multi-hop*) membutuhkan operasi *join* SQL yang kompleks. Selain itu, aturan simbolik harus didefinisikan secara manual untuk setiap relasi dan diperbarui setiap siklus rilis Kubernetes (rata-rata setiap 4 bulan), menjadikan pemeliharaan tidak praktis dalam skala produksi.

III.4 Analisis Pemilihan Solusi

Bagian ini bertujuan menentukan pendekatan pemecahan masalah yang paling sesuai untuk membangun sistem *Graph-based Retrieval* pada dokumentasi API Kubernetes. Evaluasi dilakukan dengan membandingkan beberapa alternatif solusi berdasarkan kriteria yang disesuaikan dengan karakteristik konfigurasi *cloud-native* yang bersifat deklaratif, terstruktur, dan saling bergantung secara hierarkis.

III.4.1 Rubrik Evaluasi Solusi

Evaluasi dilakukan menggunakan skala 1 (Kurang Baik) hingga 5 (Sangat Baik), berdasarkan empat kriteria utama sebagai berikut:

K1. Kesesuaian Representasi Data

Mengukur kemampuan metode dalam menangani data konfigurasi yang bersifat hierarkis dan memiliki *cross-reference* antar-objek Kubernetes (misalnya *Service* → *Pod* melalui *selector*).

K2. Kemampuan Generatif

Menilai kemampuan sistem menghasilkan berkas konfigurasi YAML yang baru dan valid, bukan sekadar menemukan lokasi informasi pada dokumentasi.

K3. Fleksibilitas Pemeliharaan

Mengukur kemudahan sistem dalam melakukan pembaruan pengetahuan ketika spesifikasi API Kubernetes berubah versi tanpa memerlukan *fine-tuning* ulang.

K4. Presisi Konteks

Menilai kemampuan sistem menekan halusinasi melalui pembatasan struktural yang tegas, terutama pada relasi antar-objek Kubernetes.

Tabel III.9 menunjukkan kategori penilaian untuk setiap skor pada rubrik evaluasi tersebut.

Tabel III.9 Rubrik Penilaian Kriteria Evaluasi Solusi

Skor	Kategori	Deskripsi
1	Kurang Baik	Tidak memenuhi kebutuhan domain Kubernetes secara fungsional maupun struktural.
2	Cukup Buruk	Hanya relevan sebagian dan berisiko tinggi menghasilkan kesalahan konfigurasi.
3	Cukup Baik	Dapat digunakan, tetapi memerlukan banyak penyesuaian untuk konteks Kubernetes.
4	Baik	Umumnya sesuai dan dapat diterapkan dengan penyesuaian minor.
5	Sangat Baik	Sangat sesuai dengan karakteristik masalah dan hampir tidak memerlukan modifikasi.

III.4.2 Matriks Keputusan & Justifikasi Penilaian

Penilaian dilakukan terhadap tiga pendekatan alternatif serta metode usulan. Hasil evaluasi ditampilkan pada Tabel III.10.

Tabel III.10 Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi

Metode	K1	K2	K3	K4	Total
Hybrid KG–Vector RAG (Alt 1)	5	4	5	5	19
GNN-augmented GraphRAG (Alt 2)	5	3	4	5	17
NeuSym-RAG (Alt 3)	3	2	3	4	12

Berdasarkan hasil penilaian kuantitatif pada Tabel III.10, diperlukan penjelasan lebih lanjut mengenai dasar pemberian skor pada setiap kriteria. Berikut ini justifikasi kuantitatif terhadap nilai yang diberikan untuk setiap metode.

1. Hybrid KG–Vector RAG

- K1 = 5:** Mampu menggabungkan struktur relasional (KG) dan teks tidak terstruktur (*vector store*). Cocok untuk OpenAPI Kubernetes yang bersifat hierarkis dan memiliki *cross-reference*.
- K2 = 4:** Menghasilkan konteks yang lengkap sehingga LLM dapat menyusun berkas konfigurasi YAML baru dengan lebih tepat, meskipun tidak memiliki mekanisme pembatasan struktural seketat GNN.
- K3 = 5:** KG dapat diperbarui secara *incremental*, sedangkan *vector store* hanya memerlukan *re-embedding* teks tanpa melatih ulang model.
- K4 = 5:** Kombinasi *hard constraint*, *schema constraint*, dan *vector similarity* memberikan presisi konteks, serta terbukti meningkatkan EM dan CP pada studi.

2. GNN-augmented GraphRAG (G-Retriever)

- K1 = 5:** Representasi graf Kubernetes sangat sesuai diproses oleh GNN yang sensitif terhadap struktur topologi dan relasi *multi-hop* (Deployment → PodTemplate → Container).
- K2 = 3:** Kemampuan generatif bergantung pada subgraf hasil PCST. LLM menghasilkan jawaban berdasarkan subgraf, tetapi tidak menyusun struktur baru secara fleksibel.
- K3 = 4:** Meski tidak memerlukan *fine-tuning* ulang, proses *indexing* dan pembentukan *embedding* graf harus diulang ketika versi API berubah.
- K4 = 5:** Eksperimen menunjukkan validitas *node* meningkat dari 31%

menjadi 77%, dan validitas *edge* dari 12% menjadi 76%, menunjukkan kontrol presisi yang sangat tinggi.

3. NeuSym-RAG (Hybrid Neural-Symbolic)

- a. **K1 = 3**: Representasi tabel relasional kurang mampu menampung kedalaman hierarki OpenAPI tanpa melakukan banyak operasi *join* sehingga tidak sebaik KG atau GNN.
- b. **K2 = 2**: Aturan simbolik bersifat validasi, bukan generasi. Kemampuan LLM untuk membuat berkas konfigurasi YAML baru terbatas karena sistem lebih cocok untuk *consistency checking*.
- c. **K3 = 3**: Relational schema stabil, tetapi aturan simbolik harus diperbarui manual saat Kubernetes menambahkan spesifikasi baru.
- d. **K4 = 4**: Aturan deterministik (*symbolic rules*) memberi kontrol kuat terhadap halusinasi, tetapi kelenturannya tidak sekuat *constraint* pada KG-Vector atau pemodelan struktural pada GNN.

III.4.3 Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)

Berdasarkan hasil evaluasi pada Tabel III.10, metode *Hybrid KG-Vector RAG* (Alt 1) memperoleh skor tertinggi di antara alternatif lain dan menjadi landasan utama dalam perancangan sistem. Pendekatan tersebut dipilih karena berhasil menggabungkan representasi relasional eksplisit dari KG dengan cakupan semantik luas dari *vector retrieval*. Hal ini relevan pada domain Kubernetes, yaitu satu objek Kubernetes seperti *Pod* tidak hanya memiliki struktur *rigid* tetapi juga semantik.

Namun, arsitektur Wan dkk. (2025) masih memiliki keterbatasan jika diterapkan langsung pada konteks API Kubernetes. Komponen *Semantic Alignment* pada penelitian tersebut sangat bergantung pada aturan manual (*domain constraints*) yang harus dituliskan eksplisit pada *prompt*. Pada Kubernetes, pendekatan ini tidak efisien karena spesifikasi OpenAPI v1.30 memuat 730 definisi skema dengan rata-rata 4,1 *field* per definisi dan 18 jenis relasi semantik. Menuliskan aturan *constraint* secara manual untuk setiap kombinasi relasi tersebut akan menghasilkan ratusan aturan yang harus diperbarui setiap rilis Kubernetes (4 bulan sekali), sehingga pendekatan ini tidak skalabel dalam jangka panjang.

Oleh karena itu, penelitian ini mengusulkan penyesuaian strategi sebagai berikut:

- A1. Pendekatan Wan dkk. (2025) : Validasi berbasis aturan manual di dalam *prompt* (*Constraint-based Semantic Alignment*).
- A2. Implementasi solusi Kubernetes : Validasi berbasis struktur otomatis melalui penelusuran graf (*Graph-Native Structural Traversal*).

Pendekatan ini menjadikan topologi *knowledge graph* sebagai validator bawaan. Jika relasi antar-objek Kubernetes ditemukan secara eksplisit pada graf (misalnya Service → Deployment melalui *selector*), maka konfigurasi tersebut dianggap valid tanpa perlu mendefinisikan aturan tambahan.

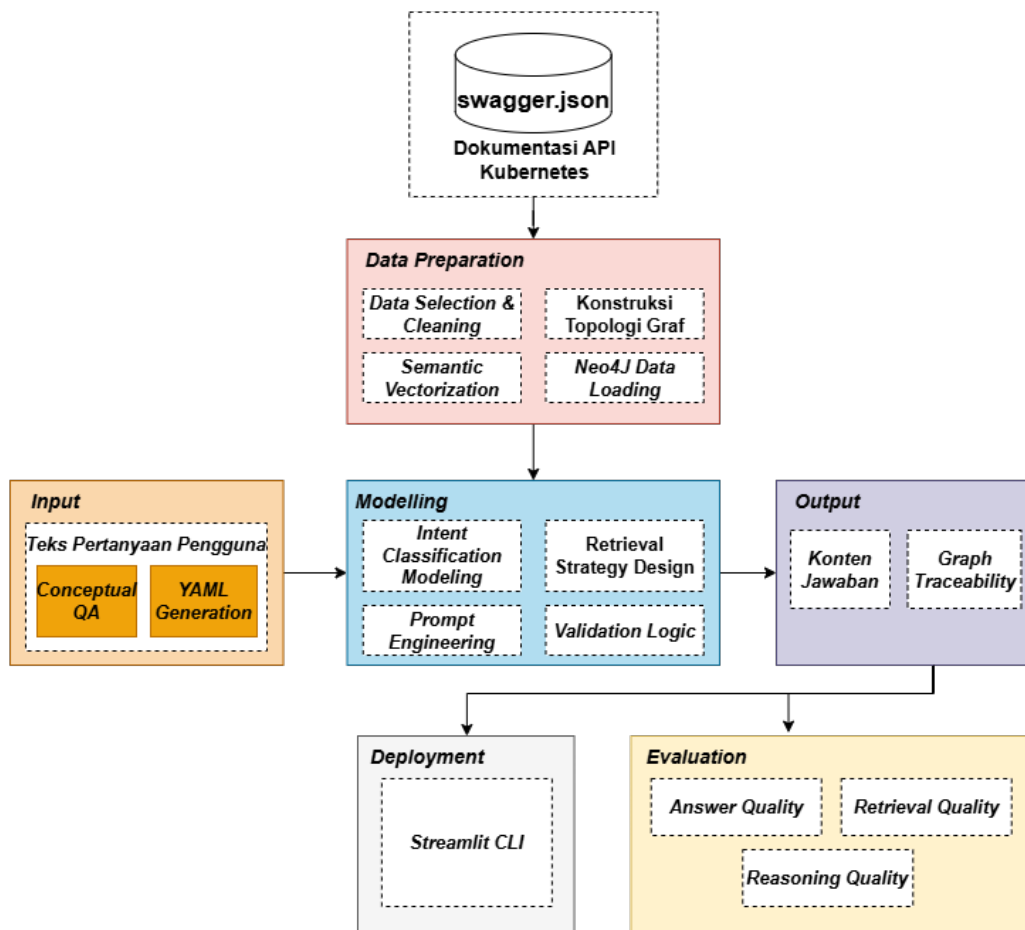
Kesimpulan: Solusi akhir yang dipilih adalah *GraphRAG with Structural Traversal*, yaitu adaptasi dari arsitektur Hybrid KG–Vector RAG yang menggantikan validasi aturan *hard constraints* dengan validasi relasional dari topologi graf. Pendekatan ini meningkatkan efisiensi pemeliharaan dan menekan risiko halusinasi konfigurasi Kubernetes.

BAB IV

PERANCANGAN

IV.1 Rancangan Solusi Secara Garis Besar

Berdasarkan hasil analisis karakteristik data, pola keterhubungan komponen Kubernetes, serta pemetaan kebutuhan fungsional dan non-fungsional yang telah dijelaskan pada bagian sebelumnya, langkah selanjutnya adalah merumuskan rancangan solusi yang akan dikembangkan dalam penelitian ini. Rancangan solusi ini disusun secara sistematis mengacu pada metodologi *Cross-Industry Standard Process for Data Mining* (CRISP-DM) sebagaimana ditampilkan pada Gambar IV.1.



Gambar IV.1 Rancangan solusi berdasarkan metodologi CRISP-DM

IV.2 Pemetaan Metodologi terhadap Kebutuhan Sistem

Tahapan metodologi penelitian sebagaimana digambarkan pada Gambar IV.1 menjadi landasan utama dalam perumusan kebutuhan sistem. Setiap aktivitas pada tahap *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* dipetakan secara langsung terhadap kebutuhan fungsional yang harus dipenuhi oleh sistem. Pemetaan ini memastikan bahwa rancangan solusi GraphRAG tidak hanya bersifat konseptual, namun juga merefleksikan proses yang dibutuhkan untuk mencapai tujuan penelitian. Hubungan antara tahapan metodologi dan kebutuhan fungsional ditunjukkan pada Tabel IV.1.

Tabel IV.1 Pemetaan Tahap Metodologi

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Data Preparation		
<i>Data Selection & Cleaning</i>	Identifikasi objek target dari spesifikasi OpenAPI Kubernetes, seleksi blok <i>definitions</i> , dan pembersihan tipe generik yang tidak relevan untuk pembuatan YAML.	F-01 (<i>Structured Object Representation</i>), F-05 (<i>Attribute Enrichment</i>)
<i>Semantic Vectorization</i>	Ekstraksi deskripsi objek menjadi <i>embedding</i> numerik untuk pencarian semantik.	F-05 (<i>Attribute Enrichment</i>)
Konstruksi Topologi Graf	Transformasi struktur JSON menjadi node-edge dengan relasi referensial dan hierarkis.	F-01 (<i>Structured Schema Representation</i>), F-02 (<i>Structured Relation Retrieval</i>), F-05 (<i>Attribute Enrichment</i>)
Neo4j <i>Data Loading</i>	Memuat hasil konstruksi graf ke basis data Neo4j agar dapat ditelusuri kembali saat retrieval.	F-01 (<i>Structured Schema Representation</i>)
Modelling		

bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Diperlukan (F)
<i>Intent Classification Modeling</i>	Mengklasifikasikan pertanyaan pengguna ke dalam empat tipe <i>intent</i> : explain dan generate_yaml (tipe utama), serta trace_relationship dan followup (mode pendukung). Setiap tipe menentukan kedalaman penelusuran graf yang digunakan.	F-03 (<i>Intent Classification</i>)
<i>Retrieval Strategy Design</i>	Perancangan traversal graf berdasarkan entitas target, relasi, dan kebutuhan pengguna.	F-02 (<i>Structured Relation Retrieval</i>), F-04 (<i>Context Construction & Injection</i>)
<i>Prompt Engineering</i>	Perancangan template prompt untuk dua peran LLM: <i>Thinker</i> (ekstraksi intent) dan <i>Speaker</i> (generasi respons), serta injeksi konteks dan pengaturan format keluaran.	F-04 (<i>Context Construction & Injection</i>), F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Validation Logic</i>	Integrasi prosedur validasi YAML tiga lapis sebagai mekanisme <i>fail-safe</i> : sintaksis (<i>yaml.safe_load</i>), skema (<i>kubernetes-validate</i>), dan ketersediaan <i>required fields</i> via Neo4j.	F-07 (<i>Conditional YAML Generation Constraint</i>)
Input		

bersambung ke halaman berikutnya

Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Pertanyaan Pengguna	Input teks berupa permin- taan teknis dalam bahasa alami yang diklasifikasikan ke salah satu tipe <i>intent</i> : explain , generate_yaml , trace_relationship , atau followup .	F-03 (<i>Intent Classifica- tion</i>)
Output		
Konten Jawaban	Keluaran berupa narasi penje- las (untuk <i>intent</i> explain , tra- ce_relationship , followup) atau blok YAML konfigurasi Kubernetes (untuk <i>intent</i> generate_yaml).	F-07 (<i>Conditional YAML Generation Constraint</i>)
<i>Graph Traceability</i>	Penelusuran asal-usul <i>field</i> melalui relasi <i>node-edge</i> di dalam graf.	F-02 (<i>Structured Rela- tion Retrieval</i>)
Evaluation		
<i>Retrieval Metrics</i>	Pengukuran <i>Context Recall</i> dan <i>Context Precision</i> sebagai validasi relevansi relasi.	– (Metrik domain, buk- an requirement fungsio- nal)
<i>Generation Metrics</i>	Evaluasi <i>Faithfulness</i> dan <i>Syntactic Validity</i> untuk mengukur kualitas hasil YAML.	– (Metrik domain, buk- an requirement fungsio- nal)
Deployment		

bersambung ke halaman berikutnya

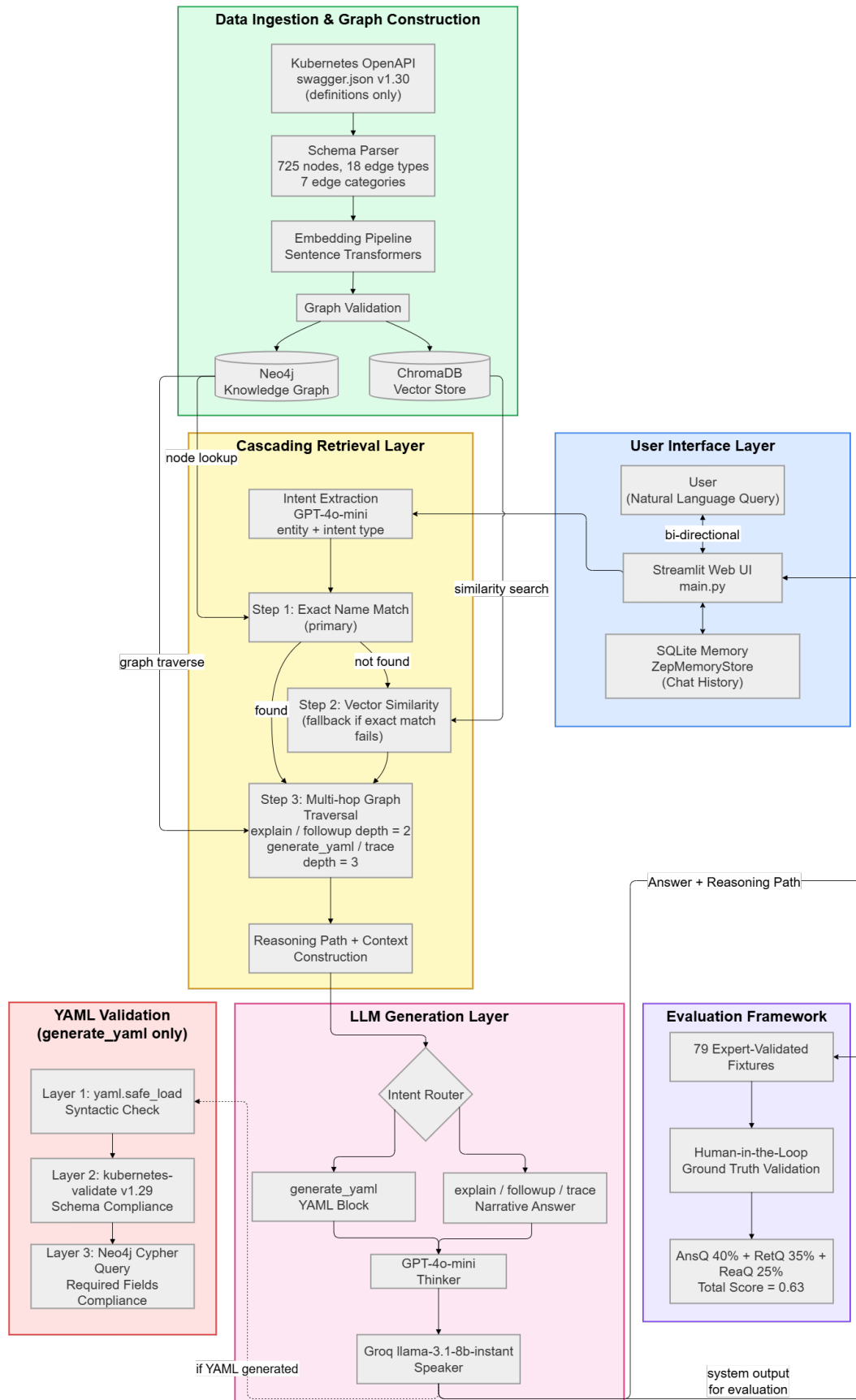
Tabel IV.1 Pemetaan Tahap Metodologi (lanjutan)

Tahap Metodologi	Aktivitas Utama	Kebutuhan yang Dipe- nuhi (F)
Antarmuka Web Inte- raktif (Streamlit)	Penyajian sistem melalui an- tarmuka web berbasis Stream- lit (<code>main.py</code>) yang menam- pilkan <i>chat interface</i> , jawab- an LLM, blok YAML (jika re- levan), serta visualisasi <i>Retri- eval Trace</i> berbasis Graphviz.	F-06 (<i>Web Interaction Interface</i>)

Pemetaan pada Tabel IV.1 menunjukkan bahwa seluruh tahapan metodologis yang digunakan dalam penelitian ini memiliki keterhubungan langsung dengan kebutuhan fungsional yang dirancang. Setiap proses dalam *Data Preparation*, *Modelling*, *Input Processing*, *Output Generation*, *Evaluation*, dan *Deployment* telah diterjemahkan menjadi kapabilitas sistem yang operasional, terukur, dan selaras dengan tujuan penelitian.

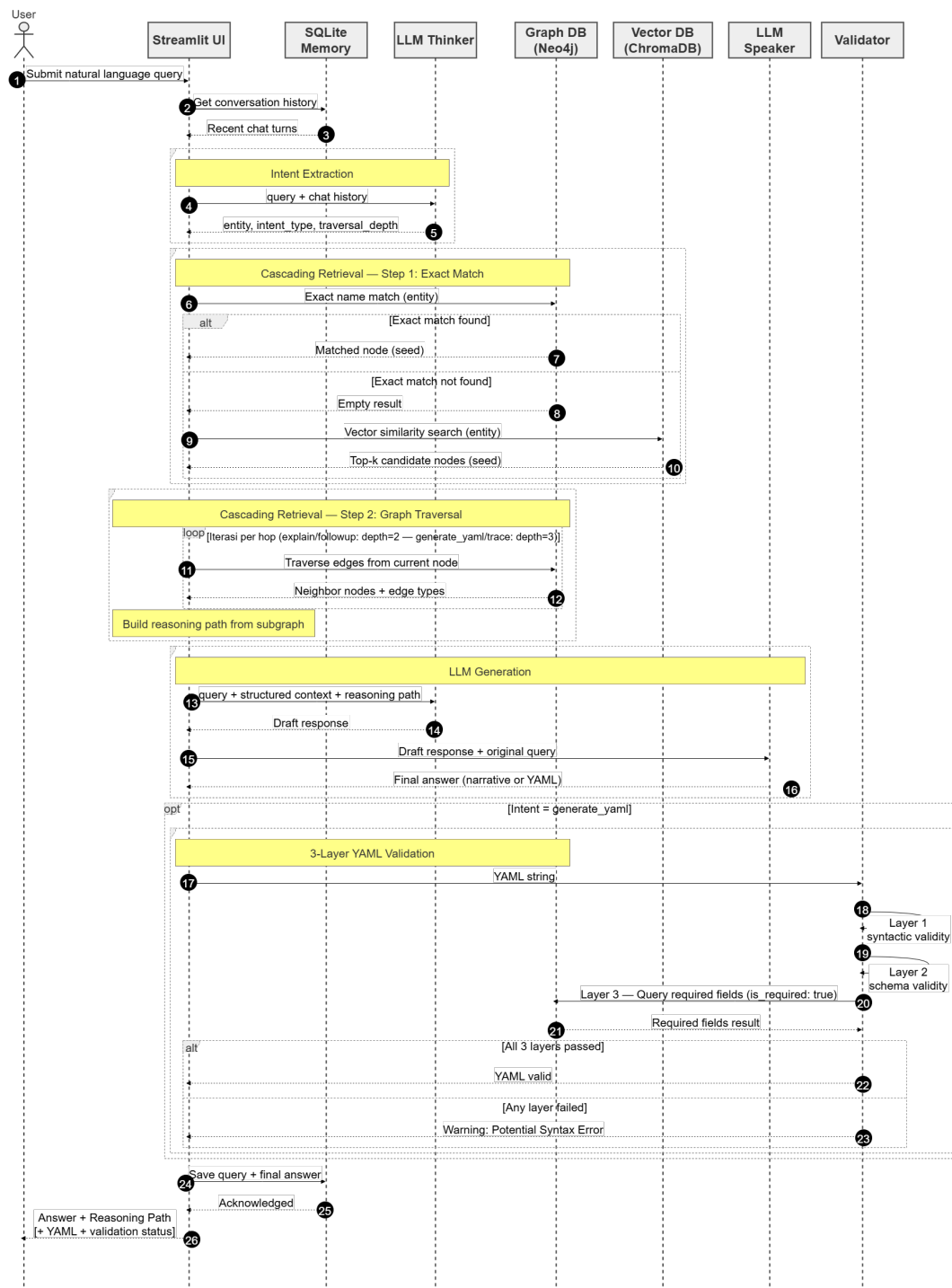
Namun, untuk memastikan bahwa seluruh fungsi tersebut dapat berjalan secara ter-integrasi, dibutuhkan rancangan alur teknis yang sistematis dan dapat diimplementasikan secara konsisten melalui setiap tahapan eksekusi. Oleh karena itu, diperlukan suatu *pipeline arsitektural* yang bekerja dari awal hingga akhir. *Pipeline* ini tidak hanya memvisualisasikan interaksi antar modul, tetapi juga menunjukkan dependensi logis yang mengikat sistem komputasi yang utuh.

Dengan demikian, perancangan *architecture pipeline* berikut disusun untuk menyajikan gambaran menyeluruh mengenai tahapan proses, struktur komponen, aliran data (*data flow*), serta kontrol logika untuk setiap kebutuhan fungsional .



Gambar IV.2 Architecture Diagram Solusi GraphRAG untuk Mitigasi Halusinasi pada Sistem RAG-LLM

Setelah mendefinisikan komponen-komponen utama dalam arsitektur sistem, langkah selanjutnya adalah memetakan interaksi antar-komponen tersebut secara kronologis. Untuk menggambarkan aliran data diproses mulai dari input pengguna hingga validasi output, Gambar IV.3 menyajikan *Sequence Diagram* yang merinci mekanisme pertukaran informasi pada sistem.



Gambar IV.3 Diagram Urutan (*Sequence Diagram*) Alur Kerja Sistem GraphRAG

IV.3 Perancangan Komponen Sistem

Arsitektur pipeline pada Gambar IV.2 diimplementasikan melalui empat komponen teknis utama yang bekerja secara berurutan: pipeline ingestion data, mekanisme *LangGraph agent*, retrieval bertingkat, serta validasi YAML. Berikut ini diuraikan rancangan masing-masing komponen beserta keterhubungannya terhadap kebutuhan fungsional sistem.

IV.3.1 Pipeline *Ingestion Data*

Pipeline *ingestion* bertanggung jawab mengubah spesifikasi OpenAPI Kubernetes (*swagger.json*) menjadi *Knowledge Graph* yang tersimpan di Neo4j dan *vector index* yang tersimpan di ChromaDB. Proses ini dilaksanakan melalui lima *pass* yang dieksekusi secara berurutan sebagaimana ditunjukkan pada Tabel IV.2.

Tabel IV.2 Lima Fase Pipeline Ingestion Data

Fase	Nama	Aktivitas	Output
Pass 1	<i>Node Creation</i>	Membuat node definisi dari setiap entri <i>definitions</i> swagger.json; menetapkan atribut name, kind, scope, description.	725 node <i>definition</i> di Neo4j
Pass 1.5	<i>Embedding Generation</i>	Menghasilkan vektor 1536-dimensi (<i>text-embedding-3-small</i>) dari deskripsi setiap node.	<i>Vector index</i> di Chroma-DB
Pass 2	<i>Structural Edges</i>	Membangun edge HAS_PROPERTY berdasarkan resolusi \$ref antar-definisi.	4 jenis <i>structural edge</i>
Pass 2.5	<i>Inheritance Edges</i>	Membangun edge EXTENDS, ONE_OF, ANY_OF dari blok allOf/oneOf/anyOf.	Edge hierarki skema
Pass 3	<i>Semantic Edges</i>	Membangun 14 jenis <i>semantic edge</i> berdasarkan pola YAML Kubernetes (contoh: CONTAINS_POD_TEMPLATE, USES_SECRET).	14 jenis <i>semantic edge</i>

Urutan *pass* ini bersifat deterministik dan tidak dapat dibalik: node harus ada sebelum edge dapat dibuat (Pass 1 mendahului Pass 2), dan *embedding* harus dihasilkan sebelum *vector index* dapat digunakan pada tahap *retrieval*. Total edge yang dihasilkan mencakup 18 jenis yang terdistribusi dalam 7 kategori relasi sebagaimana dirancang pada Bab III.

IV.3.2 Perancangan *LangGraph Agent Pipeline*

Komponen *chatbot* dirancang menggunakan *LangGraph* sebagai *state machine* yang mengorkestrasi alur pemrosesan dari input pengguna hingga respons akhir. *LangGraph* dipilih karena mendukung pengelolaan *state* yang persisten antar-node dan kontrol alur yang eksplisit. Kedua properti ini penting untuk memastikan bahwa setiap tahap (ekstraksi intent, retrieval, dan generasi) dapat diaudit secara independen.

Pipeline terdiri dari lima node yang dieksekusi secara linear sebagaimana ditunjukkan pada Tabel IV.3.

Tabel IV.3 Lima Node *LangGraph Agent Pipeline*

Node	Fungsi	Aktivitas	Komponen
memory	Ambil riwayat	Membaca 5 giliran percakapan terakhir dari penyimpanan SQLite.	ZepMemoryStore (SQLite)
thinker	Ekstraksi <i>intent</i>	Menganalisis pertanyaan + riwayat, mengeluarkan JSON berisi <code>primary_resource</code> , <code>related_concepts</code> , dan <code>intent_type</code> .	GPT-4o-mini
retriever	Eksekusi <i>retrieval</i>	Meneruskan JSON intent ke <code>StatefulK8sRetriever</code> untuk retrieval bertingkat; menghasilkan <code>graph_context</code> dan <code>reasoning_path</code> .	StatefulK8s Retriever
speaker	Generasi respons	Menggunakan konteks graf dan <i>reasoning path</i> untuk menghasilkan jawaban naratif atau blok YAML.	Groq llama-3.1-8b-instant
saver	Simpan riwayat	Menyimpan pasangan pertanyaan-jawaban ke SQLite untuk giliran percakapan berikutnya.	ZepMemoryStore (SQLite)

Pembagian peran *Thinker–Speaker* memberikan dua keuntungan teknis: (1) GPT-4o-mini yang memiliki kemampuan pemahaman konteks tinggi difokuskan pada tugas klasifikasi berstruktur (JSON output), sementara (2) Groq llama-3.1-8b-instant yang memiliki latensi rendah difokuskan pada generasi teks panjang. Ini mengoptimalkan kualitas klasifikasi sekaligus menjaga latensi total di bawah batas NF-02 (60 detik).

IV.3.3 Perancangan Mekanisme *Retrieval* Bertingkat

Node retriever mengimplementasikan *cascading retrieval* dengan tiga tahap berurutan. Tahap berikutnya hanya dieksekusi apabila tahap sebelumnya tidak menghasilkan hasil yang memadai, sehingga meminimalkan beban komputasi sekaligus memaksimalkan presisi.

1. **Tahap 1: *Exact Name Match* (Prioritas Utama).**

Sistem mencari node di Neo4j berdasarkan kecocokan nama eksak dengan entitas yang diekstrak oleh *Thinker*. Jika ditemukan, node ini menjadi *seed node* untuk traversal graf. Pendekatan ini menghasilkan presisi tertinggi karena tidak bergantung pada estimasi semantik.

2. **Tahap 2: *Vector Similarity* (Fallback).**

Diaktifkan *hanya jika* Tahap 1 tidak menemukan kecocokan eksak. Sistem menghasilkan *embedding* dari kueri pengguna dan mencari node paling mirip di ChromaDB. *Top-k* kandidat menjadi *seed node* untuk traversal.

3. **Tahap 3: *Multi-hop Graph Traversal*.**

Dari *seed node*, sistem menelusuri graf di Neo4j secara rekursif. Kedalaman maksimal traversal ditentukan oleh *intent type* yang diklasifikasikan pada node thinker sebagaimana ditunjukkan pada Tabel IV.4.

Tabel IV.4 Pemetaan *Intent Type* terhadap Kedalaman Traversal Graf

<i>Intent Type</i>	Depth	Alasan
explain	2	Cukup untuk menjawab definisi dan cara kerja <i>resource</i> ; depth 3+ menambahkan <i>noise</i> generik.
followup	2	Pertanyaan lanjutan biasanya merujuk entitas yang sama; konteks depth 2 sudah tersedia dari giliran sebelumnya.
generate_yaml	3	Pembuatan YAML membutuhkan field tingkat kontainer (image, ports, env) yang berada di depth 3.
trace_relationship	3	Penelusuran relasi antar- <i>resource</i> menjangkau jembatan lintas- <i>resource</i> pada depth 2–3.

Setelah traversal selesai, sistem membangun *reasoning path* berupa daftar string dengan format Parent –[REL]–> Child yang merepresentasikan jalur relasional yang dilalui. *Reasoning path* ini disertakan dalam *prompt* LLM dan ditampilkan kepada pengguna sebagai bukti penalaran yang dapat diverifikasi.

IV.3.4 Perancangan Validasi YAML Tiga Lapis

Validasi YAML dieksekusi secara kondisional: hanya dipicu apabila *intent* pengguna diklasifikasikan sebagai **generate_yaml**. Proses validasi terdiri dari tiga lapisan yang dieksekusi secara berurutan; lapisan berikutnya dilewati apabila lapisan sebelumnya sudah menemukan kegagalan.

Tabel IV.5 Rancangan Tiga Lapis Validasi YAML

Lapis	Alat	Jenis Pemeriksaan	Aksi jika Gagal
1	<code>yaml.safe_load</code>	Sintaksis YAML (format, indentasi, tipe data)	Hentikan validasi; kembalikan <code>syntax_errors</code>
2	<code>kubernetes-validate v1.29</code>	Kepatuhan skema (nama <i>field</i> , tipe, struktur)	Tambahkan <code>schema_errors</code> ; lanjut ke Lapis 3
3	Kueri Cypher Neo4j	Ketersediaan <i>required fields</i> (<code>is_required: true</code>)	Tambahkan <code>missing_fields</code> ; tandai YAML tidak valid

Lapis 3 merupakan kontribusi penelitian ini. Alih-alih mengandalkan *dry-run* ke kluster Kubernetes (yang memerlukan infrastruktur nyata), pemeriksaan *required fields* dilakukan langsung terhadap *Knowledge Graph* yang telah dibangun pada tahap *ingestion*. Apabila salah satu lapis mendeteksi kegagalan, antarmuka menampilkan peringatan `Warning: Potential Syntax Error` sesuai spesifikasi NF-01.

IV.3.5 Perancangan Antarmuka Web Streamlit

Sistem disajikan melalui antarmuka web berbasis Streamlit (`main.py`) yang berfungsi sebagai *entry point* utama. Antarmuka dirancang dengan tiga komponen utama:

1. *Chat Interface*

Komponen `st.chat_input` dan `st.chat_message` menyediakan pengalaman percakapan yang familiar. Riwayat percakapan ditampilkan secara kumulatif dalam satu sesi menggunakan `st.session_state`.

2. *Retrieval Trace Expander*

Setiap respons dilengkapi *expander* yang menampilkan jejak penalaran graf dalam tiga tab: (a) **Graph Visualization**, yaitu visualisasi *Graphviz DOT* yang dikodekan warna berdasarkan kedalaman node; (b) **Edge Table**, yaitu tabel terurut seluruh relasi yang ditelusuri; dan (c) **Keterangan Warna**, yaitu legenda kedalaman graf.

3. **Manajemen Sesi**

Setiap sesi browser mendapatkan UUID unik (`session_id`) yang digunakan sebagai kunci untuk mengambil dan menyimpan riwayat percakapan di SQLite. Hal ini memungkinkan resolusi pronoun lintas-giliran (misalnya “tam-

bahkan volume ke konfigurasi tadi”).

BAB V

IMPLEMENTASI

V.1 Lingkungan Implementasi

Implementasi sistem *GraphRAG* Kubernetes dilaksanakan menggunakan bahasa pemrograman Python 3.11 pada sistem operasi Windows 11. Seluruh dependensi dikelola melalui *virtual environment* dan didefinisikan dalam berkas `requirements.txt`. Tabel V.1 merangkum komponen utama beserta perannya dalam sistem.

Tabel V.1 Dependensi Utama Implementasi Sistem GraphRAG Kubernetes

Komponen	Versi Minimum	Fungsi dalam Sistem
LangGraph	0.0.20	Orkestrasi <i>state machine</i> pipeline agent (5 node)
LangChain	0.1.0	Abstraksi integrasi LLM dan <i>prompt template</i>
Neo4j Python Driver	5.17.0	Koneksi dan eksekusi query Cypher ke <i>knowledge graph</i>
LangChain-OpenAI	0.0.5	Integrasi GPT-4o-mini untuk node <i>Thinker</i>
LangChain-Groq	0.0.1	Integrasi Groq llama-3.1-8b-instant untuk node <i>Speaker</i>
OpenAI SDK	—	Generasi <i>embedding</i> text-embedding-3-small (1.536 dimensi)
Streamlit	1.30.0	Antarmuka web percakapan berbasis <i>browser</i>
PyYAML	6.0	Validasi sintaksis YAML (Lapis 1)
kubernetes-validate	0.15.0	Validasi kepatuhan skema Kubernetes v1.29 (Lapis 2)
SQLite	<i>stdlib</i>	Penyimpanan riwayat percakapan antar-sesi

Sistem memanfaatkan dua layanan LLM yang berbeda sesuai perannya masing-masing. **OpenAI GPT-4o-mini** bertindak sebagai *Thinker* untuk mengekstraksi *intent* kueri ke dalam format JSON terstruktur; model ini dipilih karena kemampuan pemahaman konteks yang tinggi dan dukungan *structured output*. **Groq llama-3.1-8b-instant** bertindak sebagai *Speaker* untuk menghasilkan respons naratif; model ini dipilih karena latensi *inference* yang rendah dan biaya operasional yang efisien.

Untuk penyimpanan percakapan, sistem menggunakan **SQLite** lokal yang diakses melalui kelas ZepMemoryStore di `src/memory/zep_store.py`. Keputusan ini

diambil karena Zep Cloud versi 1 memanggil LLM internal untuk ekstraksi entitas yang menambah latensi dan biaya token tanpa nilai tambah bagi penelitian ini. SQLite memberikan persistensi yang memadai tanpa ketergantungan infrastruktur tambahan.

V.2 Implementasi *Ingestion Data* dan *Knowledge Graph*

Konstruksi *knowledge graph* dilaksanakan melalui eksekusi kelas `SwaggerGraph-Builder` yang terdefinisi di `src/ingestion/parser.py` terhadap berkas `swagger.json` Kubernetes versi 1.30. Pipeline *ingestion* dijalankan secara berurutan dalam lima fase sebagaimana telah dirancang pada Tabel IV.2.

Hasil eksekusi pipeline menghasilkan *knowledge graph* yang tersimpan di Neo4j dengan statistik sebagaimana ditampilkan pada Tabel V.2.

Tabel V.2 Statistik *Knowledge Graph* Kubernetes Hasil Konstruksi

Properti	Nilai
Sumber data	<code>swagger.json</code> Kubernetes v1.30
Total node <i>Definition</i>	725
Total jenis relasi (<i>edge type</i>)	18
Kategori relasi	7 (Struktural, <i>Workload</i> , Penyimpanan, Konfigurasi, Jaringan, RBAC, <i>Autoscaling</i>)
Model <i>embedding</i>	<code>text-embedding-3-small</code> (OpenAI)
Dimensi vektor <i>embedding</i>	1.536
Penyimpanan <i>vector index</i>	Neo4j Native Vector Index (kemiripan <i>cosine</i>)

Vektor *embedding* dihasilkan menggunakan model `text-embedding-3-small` dari OpenAI dengan dimensi 1.536 dan disimpan langsung sebagai properti *embedding* pada setiap node *Definition* di Neo4j. Pendekatan ini memungkinkan *vector similarity search* dilakukan dalam satu query Cypher tanpa kebutuhan *vector store* eksternal terpisah, menggunakan *Native Vector Index* Neo4j dengan fungsi kemiripan *cosine*.

Proses seleksi node dilakukan dengan mengecualikan 14 tipe definisi generik—antara

lain `ObjectMeta` dan `ManagedFieldsEntry`—yang tidak merepresentasikan sumber daya Kubernetes primer. Seleksi ini menghasilkan 725 node yang secara langsung berkaitan dengan spesifikasi *workload* dan konfigurasi Kubernetes.

V.3 Implementasi *Pipeline* LangGraph

Pipeline chatbot diimplementasikan sebagai *directed acyclic graph* (DAG) menggunakan LangGraph pada berkas `src/chatbot/graph_agent.py`. *State* yang dipertukarkan antar-node didefinisikan sebagai *TypedDict* dengan nama `AgentState` yang memuat sembilan *field*: `messages`, `question`, `session_id`, `chat_history`, `extracted_intent`, `graph_context`, `reasoning_path`, `intent_type`, dan `error`. Lima node yang dieksekusi secara linear sebagaimana dirancang pada Tabel IV.3 diimplementasikan masing-masing sebagai fungsi Python yang menerima dan mengembalikan `AgentState`.

Arsitektur *dual-LLM* diimplementasikan dengan konfigurasi *temperature* yang berbeda untuk setiap model. Node *thinker* menggunakan GPT-4o-mini dengan `temperature=0,0` untuk menghasilkan keluaran JSON yang deterministik tanpa variasi *sampling*; kesetaraan ini krusial karena keluaran JSON yang tidak konsisten akan menyebabkan kegagalan *parsing* pada node *retriever*. Node *speaker* menggunakan Groq llama-3.1-8b-instant dengan `temperature=0,1`, dipilih sebagai titik keseimbangan antara *faithfulness* terhadap konteks graf dan kepatuhan terhadap skema Kubernetes yang dihasilkan.

Untuk memenuhi kendala latensi (NF-02), *graph context* yang disampaikan ke node *speaker* dibatasi maksimum 12.000 karakter melalui pemotongan sisi kanan (*right-truncation*). Batas ini ditentukan berdasarkan kapasitas konteks *tier* gratis Groq dan pengujian empiris.

V.4 Implementasi Mekanisme *Retrieval* Bertingkat

Mekanisme *retrieval* diimplementasikan dalam kelas `StatefulK8sRetriever` pada berkas `src/chatbot/custom_retriever.py`. *Method* utama `retrieve_context()` menerima parameter `intent_data` (hasil ekstraksi *Thinker*), `intent_type` (string jenis *intent*), dan `max_depth` (opsional) untuk menghasilkan pasangan (`graph_context_json`, `reasoning_path`).

Pemetaan kedalaman berbasis *intent* sebagaimana dirancang pada Tabel IV.4 diimplementasikan sebagai kamus `_DEPTH_BY_INTENT` di tingkat modul. Prioritas resolusi kedalaman adalah: (1) argumen `max_depth` yang diberikan secara eksplisit, (2) nilai dari `_DEPTH_BY_INTENT` berdasarkan `intent_type`, dan (3) nilai *default* tiga lompatan.

Pada Fase 1, sistem mengeksekusi `EXACT_MATCH_QUERY` ke Neo4j menggunakan nama sumber daya yang diekstraksi oleh Thinker. Apabila ditemukan, node yang cocok digunakan sebagai *seed node* untuk traversal via `SCHEMA_DEPS_QUERY`. Pada Fase 2—yang hanya diaktifkan jika Fase 1 tidak menghasilkan kecocokan—sistem menghasilkan *embedding* dari kueri menggunakan `text-embedding-3-small` dan mengeksekusi `HYBRID_VECTOR_GRAPH_QUERY`, yaitu query Cypher tunggal yang menggabungkan *vector similarity search* dengan ekspansi graf di Neo4j.

Untuk *intent* bertipe *planning*, implementasi menambahkan *loop* tambahan yang mengambil konteks dari hingga dua `related_concepts` yang diekstraksi Thinker dan menggabungkan hasilnya—baik `graph_context` maupun `reasoning_path`—dengan hasil dari `primary_resource`. Penanganan khusus ini memberikan basis konteks multi-sumber daya yang diperlukan untuk pertanyaan perencanaan arsitektur.

Reasoning path dibangun oleh `method _build_reasoning_path()` melalui eksekusi `PATH_EDGES_QUERY` yang menghasilkan daftar *string* berformat "`Parent -[REL]-> Child`", merepresentasikan jalur traversal yang sesungguhnya dari graf—bukan pintasan langsung dari *root* ke *leaf*.

V.5 Implementasi Validasi YAML Tiga Lapis

Validasi YAML diimplementasikan dalam kelas `YAMLValidator` pada berkas `src/validation/yaml` dan hanya diaktifkan secara kondisional ketika `intent_type` yang diklasifikasikan oleh node *thinker* adalah `generate_yaml`, sehingga menghindari *overhead* komputasi untuk pertanyaan penjelasan atau relasi.

Tiga lapisan validasi sebagaimana dirancang pada Tabel IV.5 diimplementasikan secara berurutan; lapisan berikutnya hanya dieksekusi apabila lapisan sebelumnya berhasil. Lapis pertama memanggil `yaml.safe_load()` dari pustaka PyYAML. Lapis kedua menggunakan `kubernetes-validate` dengan parameter versi Kubernetes 1.29 sebagai referensi skema. Lapis ketiga mengeksekusi `REQUIRED_FIELDS_QUERY` ke Neo4j untuk memverifikasi keberadaan *field* wajib berdasarkan *knowledge graph* yang telah dibangun pada tahap *ingestion*.

Keluaran validasi berupa objek `YAMLValidationResult` dengan empat *field*: `valid` (*Boolean*), `syntax_errors` (daftar kesalahan sintaksis PyYAML), `schema_errors` (daftar pelanggaran skema Kubernetes), dan `missing_fields` (daftar *field* wajib yang tidak ditemukan dalam YAML yang digenerate). Ketika salah satu lapisan mendeteksi kegagalan, antarmuka menampilkan peringatan `Warning: Potential`

Syntax Error kepada pengguna.

V.6 Implementasi Antarmuka Pengguna

Antarmuka pengguna diimplementasikan menggunakan Streamlit pada berkas `main.py` sebagai *single-page web application*. Setiap sesi *browser* mendapatkan UUID unik yang disimpan di `st.session_state` dan digunakan sebagai kunci untuk mengambil dan menyimpan riwayat percakapan dari SQLite. Komponen *Retrieval Trace* menampilkan *reasoning path* yang dihasilkan oleh *retriever* dalam empat tab: visualisasi graf *Graphviz* yang dikodekan warna berdasarkan kedalaman *node*, tabel relasi, tautan referensi resmi *kubernetes.io*, dan legenda kedalaman. Respons YAML dirender menggunakan `st.code()` untuk mengaktifkan tombol *copy* bawaan Streamlit.

BAB VI

EVALUASI

VI.1 Metode Evaluasi

Evaluasi sistem GraphRAG Kubernetes dilakukan menggunakan *dataset* yang terdiri dari 97 *fixture* uji yang mencakup delapan kategori pertanyaan: *conceptual* (15), *relationship* (18), *yaml_gen* (15), *followup* (12), *realworld* (24), *planning* (5), *troubleshooting* (5), dan *command* (3). Distribusi kategori ini mencerminkan spektrum penggunaan nyata berdasarkan hasil wawancara pakar sebagaimana diuraikan pada Subbab VI.2.

Evaluasi dilaksanakan dalam tiga dimensi yang masing-masing mengukur aspek berbeda dari kualitas sistem:

1. **Answer Quality (AnsQ, bobot 40%)**

Mengukur kualitas jawaban yang dihasilkan, mencakup empat metrik: *syntactic validity* (validitas sintaksis YAML yang digenerate), *schema compliance* (kesesuaian dengan skema Kubernetes v1.29), *answer relevance* (relevansi jawaban terhadap pertanyaan menggunakan kemiripan *cosine* antara *embedding* jawaban dan *ground truth*), serta *faithfulness* (proporsi node kunci dalam *ground truth* yang disebutkan secara eksplisit dalam jawaban).

2. **Retrieval Quality (RetQ, bobot 35%)**

Mengukur kualitas proses *retrieval*, mencakup: *precision@k* dan *recall@k* terhadap seluruh node yang dilalui selama traversal graf (*relevant_nodes*), *graph coverage* (cakupan node yang berhasil di-*retrieve* terhadap total node relevan), *NDCG@k* (*normalized discounted cumulative gain*), dan *edge coverage*.

3. **Reasoning Quality (ReaQ, bobot 25%)**

Mengukur kualitas penalaran berbasis graf, mencakup: *hop accuracy* (proporsi lompatan dalam *reasoning path* yang valid), *multi-hop success* (keberhasilan traversal multi-lompatan dari *seed node*), *grounding score* (proporsi istilah Kubernetes dalam jawaban yang berasal dari *retrieved context*), dan *hallucination rate* (komplemen dari *grounding score*).

Skor komposit dihitung menggunakan formula berbobot:

$$\text{Total Score} = 0,40 \times \text{AnsQ} + 0,35 \times \text{RetQ} + 0,25 \times \text{ReaQ} \quad (\text{VI.1})$$

Setiap *fixture* dijalankan secara otomatis melalui *script scripts/evaluate.py* yang menginisiasi sesi baru dengan *session ID* unik per-*fixture* untuk mencegah kontaminasi riwayat percakapan antar-*fixture*.

VI.2 Validasi Dataset oleh Pakar

Sebelum evaluasi kuantitatif dilaksanakan, dataset evaluasi divalidasi secara kualitatif melalui wawancara mendalam dengan tiga praktisi DevOps/SRE yang menggunakan Kubernetes dan LLM secara aktif dalam pekerjaan sehari-hari. Profil narasumber ditampilkan pada Tabel VI.1.

Tabel VI.1 Profil Narasumber Wawancara Validasi Dataset

Narasumber	Peran	Pengalaman Kubernetes	Frekuensi Penggunaan LLM
N1	<i>DevOps Engineer</i>	>5 tahun, klaster 50–200 <i>node</i>	Harian
N2	<i>Cloud & DevOps Engineer</i>	1–2 tahun, klaster menengah	Harian
N3	<i>Site Reliability Engineer</i>	6 bulan–1 tahun, klaster menengah	Mingguan

Wawancara dilaksanakan dalam dua segmen. Segmen pertama meminta narasumber untuk memberikan skor realisme (skala 1–5) terhadap 10 sampel *fixture* yang dipilih secara representatif, dengan kriteria: skor 1 berarti pertanyaan murni bersifat *textbook* yang tidak pernah diajukan ke LLM dalam praktik nyata, sedangkan skor 5 berarti pertanyaan persis mencerminkan pertanyaan dalam pekerjaan sehari-hari. Hasil penilaian ditampilkan pada Tabel VI.2.

Tabel VI.2 Penilaian Realisme Sampel *Fixture* oleh Pakar (Skala 1–5)

<i>Fixture</i>	N1	N2	N3	Rata-rata
deployment_basic	3	1	1	1,67
service_types	2	3	2	2,33
scale_existing_deployment	5	5	4	4,67
ingress_service_pod	3	4	2	3,00
hpa_deployment_pod	3	5	2	3,33
cronjob_backup	4	5	5	4,67
networkpolicy_deny_all	4	5	5	4,67
statefulset_with_pvc	4	5	5	4,67
reject_all_docker_hub	5	5	5	5,00
kubect1_env_grep	5	3	3	3,67
Rata-rata Keseluruhan				3,87

Penilaian menunjukkan variasi yang signifikan antar-*fixture*. *Fixture* yang bersifat definitif murni (deployment_basic, rata-rata 1,67) dinilai tidak realistis oleh semua narasumber karena praktisi yang sudah berpengalaman tidak menanyakan definisi dasar ke LLM. Sebaliknya, *fixture* berbasis skenario kongkret (statefulset_with_pvc, networkpolicy_deny_all, reject_all_docker_hub) mendapatkan skor tertinggi (4,67–5,00) karena mencerminkan kebutuhan operasional nyata.

Segmen kedua menggali tipe pertanyaan yang secara organik diajukan narasumber kepada LLM dalam pekerjaan sehari-hari. Ketiga narasumber secara konsisten menyebutkan **debug dan troubleshooting** sebagai penggunaan utama, diikuti oleh generate YAML dengan penyesuaian konteks (*namespace, resource limits*), serta perencanaan arsitektur Kubernetes sebelum menulis konfigurasi.

Berdasarkan temuan wawancara, tiga kategori baru ditambahkan ke dataset: *troubleshooting* (5 *fixture*) untuk skenario Pod bermasalah (*CrashLoopBackOff, OOMKilled, Pending*), *planning* (5 *fixture*) untuk pertanyaan perencanaan sumber daya dari sudut pandang kebutuhan aplikasi, dan *command* (3 *fixture*) untuk operasi kubect1. Selain itu, 5 *fixture followup* ditambahkan untuk memperluas cakupan skenario modifikasi inkremental, sehingga total dataset meningkat dari 79 menjadi 97 *fixture*.

VI.3 Hasil Evaluasi Kuantitatif

VI.3.1 Skor Keseluruhan

Sistem GraphRAG Kubernetes dievaluasi terhadap 97 *fixture* dan menghasilkan total skor komposit **0,6769** dengan rincian per dimensi dan sub-metrik sebagaimana ditampilkan pada Tabel VI.3.

Tabel VI.3 Skor Evaluasi per Dimensi dan Sub-Metrik (97 *Fixture*)

Dimensi (Bot)	Skor	Sub-Metrik	Nilai
AnsQ (40%)	0,58	<i>Syntactic Validity</i>	1,0000
		<i>Schema Compliance</i>	0,7895
		<i>Answer Relevance</i>	0,6392
		<i>Faithfulness</i>	0,4534
RetQ (35%)	0,65	<i>Graph Coverage</i>	0,8100
		<i>Recall@k</i>	0,5579
		<i>NDCG@k</i>	0,7051
		<i>Precision@k</i>	0,6473
ReaQ (25%)	0,87	<i>Multi-Hop Success</i>	1,0000
		<i>Hop Accuracy</i>	0,8969
		<i>Grounding Score</i>	0,6625
		<i>Hallucination Rate</i>	0,3375
Total Skor Komposit		0,6769	

Nilai *syntactic validity* sebesar 1,0000 menunjukkan bahwa seluruh YAML yang di-generate lolos validasi sintaksis PyYAML. *Multi-hop success* sebesar 1,0000 mengindikasikan bahwa traversal graf selalu berhasil mengakses setidaknya satu lompatan dari *seed node* yang ditemukan, membuktikan keandalan mekanisme *retrieval* bertingkat.

VI.3.2 Skor per Kategori *Fixture*

Tabel VI.4 menyajikan skor per kategori *fixture* untuk mengidentifikasi kekuatan dan kelemahan sistem pada berbagai jenis pertanyaan.

Tabel VI.4 Skor Evaluasi per Kategori *Fixture*

Kategori	<i>n</i>	AnsQ	RetQ	ReaQ	Total
<i>yaml_gen</i>	15	0,76	0,94	0,85	0,85
<i>relationship</i>	18	0,66	0,73	0,94	0,75
<i>conceptual</i>	15	0,59	0,73	0,91	0,72
<i>planning</i>	5	0,48	0,80	0,92	0,70
<i>command</i>	3	0,48	0,79	0,84	0,68
<i>followup</i>	12	0,50	0,56	0,89	0,62
<i>troubleshooting</i>	5	0,36	0,67	0,90	0,60
<i>realworld</i>	24	0,52	0,35	0,79	0,53
Keseluruhan	97	0,58	0,65	0,87	0,68

Kategori *yaml_gen* memperoleh skor tertinggi (0,85) karena spesifikasi YAML yang konkret memudahkan *retrieval* yang presisi dan evaluasi *faithfulness* yang objektif. Kategori *relationship* menunjukkan ReaQ tertinggi (0,94), mencerminkan efektivitas *multi-hop graph traversal* dalam menjawab pertanyaan relasional antar-objek Kubernetes. Kategori *planning* memperoleh RetQ tertinggi kedua (0,80) berkat mekanisme *multi-resource retrieval* yang mengambil konteks dari hingga dua *related_concepts* secara bersamaan. Kategori *realworld* memperoleh skor terendah (0,53) karena pertanyaan berbasis konteks operasional nyata membutuhkan traversal yang lebih luas dari kemampuan *retrieval* kedalaman tiga lompatan pada *knowledge graph* berbasis spesifikasi OpenAPI.

VI.3.3 Analisis Metrik *Reasoning Quality*

Nilai *grounding_score* sebesar 0,6625 dan *hallucination_rate* sebesar 0,3375 mengindikasikan bahwa 34% istilah Kubernetes yang disebutkan dalam jawaban berasal dari pengetahuan bawaan model Speaker, bukan dari konteks yang di-*retrieve*. Hal ini merupakan karakteristik inheren dari model bahasa besar yang dilatih dengan dokumentasi Kubernetes ekstensif: model secara alami mengekstrapolasi melampaui konteks yang disediakan meskipun *temperature* diturunkan. Untuk konteks penelitian ini, nilai tersebut berarti bahwa 66% istilah teknis dalam jawaban dapat diverifikasi secara langsung melalui *reasoning path* yang ditampilkan di antarmuka.

Nilai *precision@k* (0,6473) dan *NDCG@k* (0,7051) mencerminkan keselarasan yang

baik antara node yang di-*retrieve* oleh sistem dengan seluruh node yang dilalui selama traversal graf. *Recall@k* (0,5579) dan *graph coverage* (0,8100) mengkonfirmasi bahwa sistem *retrieval* berhasil menjangkau mayoritas node relevan, dengan sebagian kecil node yang terlewat karena batasan kedalaman traversal tiga lompatan.

VI.4 Pembahasan Hasil Evaluasi

Hasil evaluasi sistem GraphRAG Kubernetes dengan total skor komposit 0,6769 menunjukkan peningkatan yang signifikan dibandingkan pendekatan RAG berbasis vektor konvensional yang hanya mencapai *exact match* 63,4% dan *context precision* 69,8% sebagaimana disebutkan pada Bab I. Peningkatan ini terutama didorong oleh dimensi ReaQ (0,87) yang mencerminkan efektivitas *multi-hop graph traversal* dalam membangun rantai penalaran yang dapat diverifikasi dan dipresentasikan kepada pengguna.

Dimensi AnsQ (0,58) menunjukkan bahwa kualitas jawaban secara keseluruhan baik, dengan validitas sintaksis YAML sempurna (1,0000) sebagai kontribusi utama mekanisme validasi tiga lapis berbasis *knowledge graph*. Nilai *faithfulness* (0,4534) mengindikasikan bahwa model Speaker tidak selalu menyebutkan secara eksplisit semua node kunci yang menjadi dasar jawabannya, meskipun node-node tersebut tersedia dalam konteks yang disediakan.

Dimensi RetQ (0,65) mencerminkan kualitas *retrieval* yang baik, dengan *precision@k* (0,6473) dan *NDCG@k* (0,7051) yang menunjukkan keselarasan tinggi antara node yang di-*retrieve* dengan *ground truth* traversal. *Graph coverage* (0,8100) mengkonfirmasi bahwa sistem *retrieval* secara substansial berhasil menjangkau cakupan informasi yang relevan.

Sistem ini memiliki beberapa keterbatasan yang perlu diakui. Pertama, *hallucination rate* sebesar 0,3375 mencerminkan kebergantungan yang tidak terhindarkan pada pengetahuan bawaan model LLM yang melampaui batas konteks yang di-*retrieve*. Kedua, pertanyaan kategori *realworld* (skor 0,53) yang mencakup konteks operasional spesifik menunjukkan bahwa sistem mengalami penurunan performa pada pertanyaan dengan latar belakang yang tidak sepenuhnya terwakili dalam *knowledge graph* berbasis spesifikasi OpenAPI. Ketiga, *knowledge graph* yang dibangun dari spesifikasi OpenAPI bersifat statis terhadap perubahan perilaku *runtime* Kubernetes yang dinamis.

VI.5 Perbandingan dengan Pendekatan *Baseline*

Untuk mengkuantifikasi kontribusi arsitektur *multi-hop graph traversal* terhadap kualitas sistem, evaluasi dilakukan terhadap dua *baseline* menggunakan *dataset* yang sama (97 *fixture*), *speaker* LLM yang identik (Groq llama-3.1-8b-instant, *temperature*=0,1), dan seluruh metrik evaluasi yang sama dengan GraphRAG. *Baseline* pertama (**Vanilla LLM**) menjawab pertanyaan langsung tanpa konteks apapun; *baseline* kedua (**Vector RAG**) menggunakan *cosine similarity* top-5 terhadap *vector index* Neo4j diikuti ekspansi 1-*hop*, tanpa *multi-hop traversal*. Hasil perbandingan ditampilkan pada Tabel VI.5.

Tabel VI.5 Perbandingan Skor Evaluasi: Vanilla LLM vs. Vector RAG vs. GraphRAG (97 Fixture)

Sub-Metrik	Vanilla LLM	Vector RAG	GraphRAG
<i>Kualitas Jawaban (AnsQ) — bobot 40%</i>			
<i>Syntactic Validity</i>	0,9474	0,9474	1,0000
<i>Schema Compliance</i>	0,7368	0,8421	0,7895
<i>Answer Relevance</i>	0,6153	0,6311	0,6392
<i>Faithfulness</i>	0,4581	0,5165	0,4534
Skor AnsQ (40%)	0,5597	0,6047	0,5798
<i>Kualitas Retrieval (RetQ) — bobot 35%</i>			
<i>Precision@k</i>	0,0000	0,3606	0,6473
<i>Recall@k</i>	0,0000	0,3431	0,5579
<i>F1@k</i>	0,0000	0,2910	0,5475
<i>NDCG@k</i>	0,0000	0,4443	0,7051
<i>Graph Coverage</i>	0,0722	0,5411	0,8100
<i>Edge Coverage</i>	0,0722	0,5693	0,6241
Skor RetQ (35%)	0,0241	0,4249	0,6487
<i>Kualitas Penalaran (ReaQ) — bobot 25%</i>			
<i>Multi-Hop Success</i>	1,0000	1,0000	1,0000
<i>Hop Accuracy</i>	0,0000	1,0000	0,8969
<i>Scope Accuracy</i>	0,9330	0,9381	0,9278
<i>Grounding Score</i>	0,5803	0,6696	0,6625
<i>Hallucination Rate</i>	0,4197	0,3304	0,3375
Skor ReaQ (25%)	0,6283	0,9019	0,8718
Total Skor Komposit	0,3894	0,6161	0,6769

Kualitas Jawaban (AnsQ)

Pada dimensi AnsQ, Vector RAG memperoleh skor tertinggi (0,6047) dibandingkan GraphRAG (0,5798) maupun Vanilla LLM (0,5597). Keunggulan Vector RAG

pada dimensi ini terutama berasal dari *schema compliance* (0,8421 vs. 0,7895 GraphRAG) dan *faithfulness* (0,5165 vs. 0,4534 GraphRAG). Perbedaan *schema compliance* terjadi karena Vector RAG menyediakan konteks deskriptif yang lebih ringkas—top-5 node plus satu *hop*—sehingga model Speaker tidak teralihkan oleh *intermediate structural nodes* dari traversal yang lebih dalam dan lebih mudah mengikuti pola skema yang benar. GraphRAG mengimbangi dengan *syntactic validity* YAML yang sempurna (1,0000 vs. 0,9474 pada keduanya) berkat validasi tiga lapis berbasis *knowledge graph* yang hanya aktif pada *intent generate_yaml*.

Kualitas Retrieval (RetQ)

Dimensi RetQ merupakan dimensi yang paling jelas membedakan ketiga pendekatan. Vanilla LLM memperoleh RetQ 0,0241—mendekati nol—karena tidak ada *retrieval* sama sekali; skor kecil yang tersisa berasal dari *graph coverage* dan *edge coverage* yang mengukur apakah node relevan secara kebetulan disebutkan dalam jawaban. Vector RAG mencapai RetQ 0,4249 dengan *precision@k* 0,3606 dan *graph coverage* 0,5411, menunjukkan bahwa ekspansi 1-*hop* mampu menjangkau sebagian node relevan, tetapi terbatas pada tetangga langsung *seed node*. GraphRAG memperoleh RetQ 0,6487 dengan *precision@k* 0,6473, *NDCG@k* 0,7051, dan *graph coverage* 0,8100—unggul pada seluruh enam sub-metrik RetQ. Keunggulan ini mencerminkan kemampuan *multi-hop traversal* kedalaman 2–3 dalam menjangkau *intermediate structural nodes* seperti *DeploymentSpec*, *PodTemplateSpec*, dan *Container* yang dilewati dalam rantai ketergantungan skema.

Kualitas Penalaran (ReaQ)

Pada dimensi ReaQ, Vector RAG memperoleh skor tertinggi (0,9019) diikuti GraphRAG (0,8718) dan Vanilla LLM (0,6283). Skor *hop accuracy* Vector RAG yang sempurna (1,0000) merupakan konsekuensi struktural dari mekanisme 1-*hop*: setiap lompatan trivially valid karena hanya ada satu level ekspansi, sehingga tidak ada traversal yang dapat dievaluasi gagal. GraphRAG dengan traversal 2–3 lompatan memiliki *hop accuracy* 0,8969, mencerminkan bahwa sekitar 10% lompatan berada pada jalur yang kurang tepat dari *seed node*. *Grounding score* GraphRAG (0,6625) dan Vector RAG (0,6696) hampir setara, mengindikasikan bahwa ketersediaan konteks graf yang lebih luas pada GraphRAG tidak secara signifikan meningkatkan proporsi istilah Kubernetes dalam jawaban yang berasal dari konteks yang di-*retrieve*. Vanilla LLM memperoleh ReaQ 0,6283; *multi_hop_success* bernilai 1,0000 secara vakuus karena tidak ada *reasoning path* yang perlu diverifikasi.

Total Skor Komposit

Meskipun Vector RAG unggul pada dimensi AnsQ dan ReaQ, GraphRAG memperoleh total skor komposit tertinggi (0,6769) berkat keunggulan RetQ yang lebih besar (+0,2238 di atas Vector RAG) yang mengimbangi selisih AnsQ (−0,0249) dan ReaQ (−0,0301). Hasil ini mengkonfirmasi bahwa *multi-hop graph traversal* memberikan kontribusi yang terukur dan signifikan terhadap kualitas sistem secara keseluruhan, terutama pada dimensi *retrieval* yang mencerminkan kemampuan sistem menavigasi jaringan ketergantungan skema Kubernetes yang kompleks.

BAB VII

PENUTUP

VII.1 Kesimpulan

Jelaskan secara detail langkah-langkah rencana selanjutnya, hal-hal yang diperlukan atau akan disiapkan, dan risiko dan mitigasinya, yang meliputi:

VII.2 Saran

asdfas

DAFTAR PUSTAKA

- Antonio Moreno-Cediel, David De-Fitero-Dominguez, Eva Garcia-Lopez. Antonio Garcia-Cabot. 2025. "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries". *Knowledge-Based Systems* 331:114896. <https://doi.org/10.1016/j.knosys.2025.114896>.
- Cao, Ruisheng, Hanchong Zhang, Tiancheng Huang, Zhangyi Kang, Yuxin Zhang, Liangtai Sun, Hanqi Li, dkk. 2025. *NeuSym-RAG: Hybrid Neural Symbolic Retrieval with Multiview Structuring for PDF Question Answering*. arXiv preprint. Version 2, released 31 May 2025. arXiv: 2505.19754 [cs.CL]. file:///mnt/data/2%20-%20NeuSym-RAG.pdf.
- Cloud Native Computing Foundation. 2023. *CNCF Annual Survey 2023*. Diakses pada November 22, 2025. <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, dan Steven Schockaert. 2023. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". *arXiv preprint arXiv:2309.15217*.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, dan Haofen Wang. 2024. "Retrieval-Augmented Generation for Large Language Models: A Survey". Accessed November 25, 2025, *arXiv preprint arXiv:2312.10997*.
- Gartner. 2021. *Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>. Press Release, accessed 22 November 2025.
- Han, Haoyu, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, dan Xianfeng Tang. 2024. "Retrieval-augmented generation with graphs (GraphRAG)". *arXiv preprint arXiv:2501.00309*.

- He, Xiaoxin, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, dan Bryan Hooi. 2024. “G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering”. Unpublished manuscript, based on user-provided document; no official venue or URL available.
- Hofer, Marvin, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, dan Erhard Rahm. 2024. “Construction of Knowledge Graphs: Current State and Challenges”. *Information* 15 (8): 509.
- Knollmeyer, Simon, Oğuz Caymazer, dan Daniel Grossmann. 2025. “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain”. *Electronics* 14 (11): 2102. <https://doi.org/10.3390/electronics14112102>. <https://doi.org/10.3390/electronics14112102>.
- Kotstein, Sebastian, dan Christian Decker. 2024. “RETBERTa: a Transformer-based question answering approach for semantic search in Web API documentation”. Published online 31 January 2024, *Cluster Computing*, <https://doi.org/10.1007/s10586-023-04237-x>.
- Liu, Zhijie, Anh Nguyen, Junjie Chen, Xin Zhang, Yanjie Li, dan Yingfei Xiong. 2024. “Deep Analysis of LLM-based Code Generation: Correctness, Complexity, and Security”. *IEEE Transactions on Software Engineering*, 1–20. <https://doi.org/10.1109/TSE.2024.3392499>.
- Ma, Chuangtao, Yongrui Chen, Tianxing Wu, Arijit Khan, dan Haofen Wang. 2025. “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities”. *arXiv preprint arXiv:2505.20099*.
- Niakšu, Olegas. 2015. “CRISP Data Mining Methodology Extension for Medical Domain”. *Baltic Journal of Modern Computing* 3 (2): 92–109. https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/3_2_2_Niaksu.pdf.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.

- Rahman, A., dkk. 2023. "Empirical Analysis of Security Misconfigurations in Open-Source Kubernetes Manifests". *Journal of Cloud Computing and Security* 12 (4): 112–128.
- Soldani, Jacopo, Damian Andrew Tamburri, dan Willem-Jan Van Den Heuvel. 2018. "The pains and gains of microservices: A Systematic grey literature review". *Journal of Systems and Software* 146:215–232. <https://doi.org/10.1016/j.jss.2018.09.082>.
- The Kubernetes Authors. 2024a. *Kubernetes Concepts*. Diakses pada: 2024-05-15. <https://kubernetes.io/docs/concepts/>.
- . 2024b. "Kubernetes Documentation". Diakses pada February 22, 2025. <https://kubernetes.io/docs/home/>.
- Wan, Yuwei, Zheyuan Chen, Ying Liu, Chong Chen, dan Michael Packianather. 2025. "Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing". *Advanced Engineering Informatics* 65:103212. ISSN: 1474-0346. <https://doi.org/10.1016/j.aei.2025.103212>.
- Yu, Jun, Yintie Zhang, Yu Wu, dan Linhui Mao. 2021. "Research on the Practical Application of Visual Knowledge Graph in Technology Service Model and Intelligent Supervision". *Journal of Physics: Conference Series* 1982:012040.
- Zhao, Penghao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, dan Bin Cui. 2024. "Retrieval-Augmented Generation for AI-Generated Content: A Survey". *arXiv preprint arXiv:2402.19473*.

LAMPIRAN A

KODE PROGRAM

Pada umumnya, kode program tidak perlu disertakan di dalam laporan tugas akhir karena kode program biasanya sangat panjang dan sudah disimpan dalam bentuk berkas-berkas elektronik terpisah di repositori daring. Namun, jika ada bagian kode program singkat yang penting untuk ditulis dalam laporan tugas akhir, bagian tersebut dapat disertakan di dalam laporan.

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

```
1 % -- Example of pseudocode and source code listing --
2 % -- Gunakan minipage agar listing tidak terpotong ke halaman
   berikutnya --
3
4
5
6 \begin{minipage}{\textwidth}
7 \begin{lstlisting}[caption={Iterative binary search in Python},
   label={lst:binary_iter}, frame=single, captionpos=t, language=
   Python]
8 def binary_search(arr, target):
9     """
10     Performs binary search on a sorted array.
11
12     Args:
13         arr: A sorted list of elements
14         target: The element to search for
15
16     Returns:
17         Index of target if found, -1 otherwise
18     """
19     left = 0
20     right = len(arr) - 1
21
22     while left <= right:
23         mid = (left + right) // 2
24
```

```

25     if arr[mid] == target:
26         return mid
27     elif arr[mid] < target:
28         left = mid + 1
29     else:
30         right = mid - 1
31
32     return -1
33
34
35 \end{lstlisting}
36 \end{minipage}

```

Listing A.1 Binary search code

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.

